# REVISE FOR FS 1

All questions and answers, copied exactly as written in every source file.

121 Java files • 20 topics • 7681 lines of code

Source folder: code_vault

13 November 2026

# Contents

**fds deeps . . . . . 6**

chillar_if_else.java . . . . . 6

Contains_Duplicate_217.java . . . . . 7

fds19.java . . . . . 9

fds20.java . . . . . 9

fds21.java . . . . . 9

fds22.java . . . . . 10

fds23.java . . . . . 11

fds24.java . . . . . 12

fds26.java . . . . . 13

fds27.java . . . . . 15

fds28.java . . . . . 15

fds32.java . . . . . 17

fds5.java . . . . . 17

First_Unique_Character_in_a_String_387.java . . . . . 18

lamda_func.java . . . . . 19

left_rotation.java . . . . . 19

Longest_Consecutive_Sequence_128.java . . . . . 20

Longest_Substring_Without_Vowels.java . . . . . 21

Reverse_Array_In_Groups .java . . . . . 21

Reverse_Words_in_a_String_151.java . . . . . 23

Reverse_Words_in_a_String_III.java . . . . . 23

right_rotation_189.java . . . . . 24

Rotate_Array_189_3.java . . . . . 24

rotate_array_796.java . . . . . 25

rotate_array_796_2.java . . . . . 26

streams.java . . . . . 27

Valid_Anagram_242.java . . . . . 27

Valid_Anagram_Application_242.java . . . . . 29

Word_Pattern_290.java . . . . . 30

Word_Pattern_290_1.java . . . . . 31

Word_Pattern_290_2.java . . . . . 33

zeroes_to_end.java . . . . . 35

**may_holidays / arrays . . . . . 37**

ques1.java . . . . . 37

ques10.java . . . . . 38

ques2.java . . . . . 39

ques3.java . . . 40
ques4.java . . . 41
ques5.java . . . 42
ques6.java . . . 42
ques7.java . . . 42
ques8.java . . . 43
ques9.java . . . 44

**may_holidays / collectionframeworks . . . 46**

ques1.java . . . 46
ques2.java . . . 46

**may_holidays / recurssion . . . 47**

ques1.java . . . 47
ques2.java . . . 47
substring.java . . . 48

**may_holidays / sliding window . . . 49**

ques1.java . . . 49
ques2.java . . . 49
ques3.java . . . 50
tempCodeRunnerFile.java . . . 50

**may_holidays / streams . . . 51**

ques1.java . . . 51

**may_holidays / strings . . . 52**

ques1.java . . . 52
ques2.java . . . 54
ques3.java . . . 54
ques4.java . . . 55
ques5.java . . . 56
ques6.java . . . 57
ques7.java . . . 58
ques8.java . . . 59

**may_holidays / two ponters . . . 61**

ques1.java . . . 61
rev.java . . . 61
water.java . . . 62

**questions_from_world . . . 64**

1.java . . . 64
Answer_Queries_with_Sum_of_Elements_2389.java . . . 64
Sort_Array_By_Parity_905.java . . . 65

**striver / arrays / faq_medium . . . 66**

leaders_in_a_array.java . . . 66
Pascals_Triangle_3.java . . . 67
Pascals_Triangle_I.java . . . 68
Rearrange_by_Sign.java . . . 69
Rotate_Matrix_by_90_Degrees.java . . . 70
Set_Matrix_Zeroes.java . . . 71
spiral_matrix.java . . . 72

**striver / basicmaths . . . 74**

gcd.java . . . 74
primeno.java . . . 75

**striver / binary_search / 2d array . . . 76**

Find_Row_with_Maximum_1s.java . . . 76

**striver / binary_search / fundamentals . . . 77**

bs.java . . . 77
lowerbound.java . . . 77
Search_in_a_2D_Matrix.java . . . 77
upperbound.java . . . 78

**striver / binary_search / logic_building . . . 79**

Count_Occurrences_in_a_Sorted_Array.java . . . 79
Find_minimum_in_Rotated_Sorted_Array.java . . . 79
Find_Number_of_Times_Array_is_Rotated.java . . . 80
First_and_last_occurrence.java . . . 81
Floor_and_Ceil_in_Sorted_Array.java . . . 82
Kth_Missing_Positive_Number.java . . . 83
Search_in_rotated_sorted_array_I.java . . . 84
Search_in_Rotated_Sorted_Array_II.java . . . 85
search_insert_pos.java . . . 85
Single_Element_in_Sorted_Array.java . . . 86

**striver / binary_search / on_answers . . . 88**

Capacity_to_Ship_Packages_Within_D_Days.java . . . 88
Find_Nth_Root_of_a_Number.java . . . 89

Find_Square_Root_of_a_Number.java . . . 90
Find_the_Smallest_Divisor.java . . . 91
Koko_Eating_Bananas.java . . . 93
Minimum_Days_to_Make_M_Bouquets.java . . . 93

**striver / greedy / easy . . . 96**

Assign_Cookies.java . . . 96
Fractional_Knapsack.java . . . 97
Jump_Game_1.java . . . 98
Lemonade_Change.java . . . 99

**striver / greedy / secduling and intervaling . . . 101**

Candy.java . . . 101
Job_Sequencing_Problem.java . . . 103
N_Meetings.java . . . 105
Non_Overlapping_Intervals.java . . . 106
Shortest_Job_First.java . . . 106

**striver / patterns . . . 108**

p6.java . . . 108
pattern1.java . . . 108
pattern2.java . . . 108
pattern3.java . . . 109
pattern4.java . . . 109
pattern5.java . . . 110

**striver / rec . . . 111**

factorial.java . . . 111
printlinearly.java . . . 111
printlinearlybacktracking.java . . . 111
printname.java . . . 112
printulta.java . . . 112
printulta2.java . . . 113
reverse_of_array.java . . . 113
sum_of_nterms.java . . . 113

**striver / sortings . . . 115**

bubblesort.java . . . 115
selectionsort.java . . . 115

# fds deeps

### chillar_if_else.java

```
// ================================================================
// Continuing our journey in mastering conditional statements & our interest with cricket.
// Let's help the Indian cricket's governing body (BCCI) to automate its plan of allotting pensions to former
players.

// Rules:
// •    If a player has played more than 10 test matches AND more than 100 ODIs → Rs. 50,000
// •    If a player has played more than 10 test matches → Rs. 25,000
// •    If a player has played more than 100 ODIs → Rs. 15,000
// •    If a player has played for India → Rs. 10,000
// •    Amount is incremented by 1/4th for every 'Man of the Match' award
// •    If not played for India but played IPL → Rs. 8,000
// •    If not played for India nor IPL → Rs. 7,000

// Sample Input and Output 1:
// Has the player represented for India[y or n]?
// y
// Enter the number of Test matches he has played
// 15
// Enter the number of ODI matches he has played
// 101
// Number of man of the matches he has won
// 2
// Amount received as pension: Rs.75000.0

// Sample Input and Output 2:
// Has the player represented for India[y or n]? n
// Has he played IPL[y or n]? y
// Amount received as pension: Rs.8000.0

// Concept: Conditional Logic (nested if-elif-else) with arithmetic
import java.util.*;
public class chillar_if_Else{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        System.out.println("Has the player represented for India[y or n]?");
        char india = sc.next().charAt(0);

        double pension = 0;

        if (india == 'y'){

            System.out.println("Enter the number of Test matches he has played");
            int tests = sc.nextInt();

            System.out.println("Enter the number of ODI matches he has played");
            int odis = sc.nextInt();

            System.out.println("Number of man of the matches he has won");
            int mom = sc.nextInt();

            if (tests > 10 && odis > 100) {
                pension = 50000;
         } else if (tests > 10){
                pension = 25000;
}       else if (odis > 100){
                pension = 15000;
} else{
                pension = 10000;
}
            pension += pension * 0.25 * mom;

}        else{
            System.out.println("Has he played IPL[y or n]?");
            char ipl = sc.next().charAt(0);
            if (ipl == 'y' || ipl == 'Y') {
                pension = 8000;
}           else {
                pension = 7000;
}

}
      System.out.println("Amount received as pension: Rs." + pension);
        sc.close();


}
```

### Contains_Duplicate_217.java

```
/*
Imagine you are building a University Exam Registration System.
Each student is assigned a unique registration ID stored in an integer array nums.
Due to a system glitch, some IDs might be recorded more than once.
Your task is to check whether any registration ID appears more than once.
If any duplicate exists → print true
If all IDs are unique → print false

Problem Statement
Given an integer array nums, return:
true if any value appears at least twice
false if every element is distinct

Input Format
nums = [element1,element2,element3,...]

Input is given in single line
Elements are comma-separated
Values may be positive or negative

Output Format
true

or
false

Constraints
1 <= nums.length <= 10^5
-10^9 <= nums[i] <= 10^9

 Large input size possible
 Negative numbers allowed
 Large integer values allowed

Sample Test Cases
Test Case1
input=[-5,-4,-3,-2,-1,0,1,2,3,4]
output=false
Test Case2
input=nums = [0,0,1,2]
output=true

*/
// brinda approach

/*import java.util.*;
class fds16
{
    public static void main(String args[])
    {
       Scanner s=new Scanner(System.in);

        String a=s.nextLine();
        a=a.replace("[","").replace("]","");
        String b[]=a.split(",");
        int c=0;
        for(int i=0;i<b.length;i++)


{



            for(int j=i+1;j<b.length;j++)


{


                if(Integer.parseInt(b[i])==Integer.parseInt(b[j]))


{


                    c++;


}
```

```
}


}

        if(c>=1)


{


                System.out.println(true);


}
        else


{


                System.out.println(false);


}

}

}*/
 // my appraoch
 import java.util.*;
public class Contains_Duplicate_217{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 [] = s.nextLine().split(",");
        Integer l [] = new Integer[s1.length];
        for(int i=0;i<s1.length;i++){
            l[i]= Integer.parseInt(s1[i]);
        }
        HashMap <Integer, Integer> m = new HashMap<>();
        for (int i : l){
            m.put(i,m.getOrDefault(i,0)+1);
        }
        for (int i : m.values()){
            if (i > 1) {
                System.out.println("true");
                return ;
            }
        }
        System.out.print("false"); // em lekapthey idi varaku vastasdi kada

    }
}
// Ai appraoch :
/*import java.util.*;
class fds16 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        String[] input = s.nextLine().split(",");

        Set<Integer> seen = new HashSet<>();
        for(String numStr : input){
            int num = Integer.parseInt(numStr.trim());
            if(seen.contains(num)){
                System.out.println("true"); // duplicate found
                return;
            }
            seen.add(num);
        }
        System.out.println("false"); // no duplicates
    }
} */
/* // sort and adjacent check
        Arrays.sort(arr);

        for (int i = 0; i < n - 1; i++) {
            if (arr[i] == arr[i + 1]) {      // equal values are adjacent after sorting
                System.out.println("true");
                return;
            }
        }
```

```
        System.out.println("false");
    }
} */
```

### fds19.java

```
import java.util.*;

class fds19{
    public static void main(String[] args) {
        String s = "goodmorning";
        String [] k = s.split("");
        //System.out.print(k[5]);
        int l=0,r=0;
        Set <String> s1 = new HashSet<>();
        while(r<k.length && l<k.length){
            if(s1.contains(k[r])){
                l++;
            }
            else{
                s1.add(k[r]);
                r++;
            }
        }
        System.out.print(r);
    }
}
```

### fds20.java

```
//longest substring

import java.util.HashSet;
class fds20{
   public static void main(String[] args) {
        String k = "baca";
        HashSet <Character> h = new HashSet<>();
        int i = 0;
        int l = 0,r = 0,maxlen = 0;
        while (r<k.length()){ //r munduku potunndi ga
            while(h.contains(k.charAt(r))){  // to remove all elements before it , while vadutunnam
                h.remove(k.charAt(l));
                l++;
            if(!h.contains(k.charAt(r))){;
                h.add(k.charAt(r));
                if(r-l+1>maxlen){
                    maxlen = r - l + 1;
                    i = l;
                }
                r++;
}
        }
String k1 = k.substring(i,i+maxlen);
System.out.println(k1);
}}
}
```

### fds21.java

```
/*Given strings s and t, find the minimum window in s that contains all characters of t (including duplicates).
If no such window exists, return empty string.
Input:  s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"

Input:  s = "a", t = "aa"
Output: ""

Hint:
Use a HashMap for character counts in t.
Use two pointers and a need/have counter. Shrink left pointer when window is valid to minimise length.
*/
public class fds21 {

}
```

### fds22.java

```
/*
Imagine you are developing a digital newspaper system.
A user wants to create a custom news headline (ransomNote)
using letters available from a printed newspaper page (magazine).
Rules:
Each letter from the magazine can be used only once.
You must determine whether the headline can be constructed using available letters.
If possible → return true
Otherwise → return false

Problem Statement

Given two strings:
ransomNote
magazine

Return true if ransomNote can be constructed using letters from magazine.
Otherwise, return false.

Each character in magazine can only be used once.

 Input Format:
First line  : ransomNote
Second line : magazine
Output Format:
Print true or false

Constraints:
1 <= ransomNote.length, magazine.length <= 10^5
Both strings consist of only lowercase English letters.

Sample Test Cases
Testcase1
input=aaa
aa
output=false

Testcase2
input=abc
cba
output:true

*/
import java.util.*;
class fds22{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String ran = s.nextLine();
        String mag = s.nextLine();
        //int res =freqarray_apporach(mag,ran));
        int res = freqmap_approach(mag,ran);
        System.out.println(res == 1);   // direct ga function lo kuda print cheyaachu
        //but oorikey show kosam converting int results into boolean
    }
   public static int freqarray_apporach(String mag,String ran){
        mag =mag.toLowerCase();
        ran = ran.toLowerCase();
        if(mag.length()<ran.length()){
            return 0;                    // see fds book
        }
        int freq [] = new int [26];// freq array , java by default initilizes with zero kada
        for (char c : mag.toCharArray()){
            freq[c-'a']++; // see in fds note book

        }
        for (char c: ran.toCharArray()){
            freq[c-'a']--;

        }
        for (int i=0;i<freq.length;i++){
            if(freq[i]<0)
            return 0;
        }
        return 1;
    }
    public static int freqmap_approach(String mag,String ran){
        mag = mag.toLowerCase();
        ran = ran.toLowerCase();
        if(mag.length()<ran.length())
        return 0;
```

```
        Map <Character, Integer> m = new HashMap<>();
        for (char c : mag.toCharArray()){
            //m.put(c,c.getOrDefault(c,0)+1); okavela idhi marchipothey try this noob way
            if(m.containsKey(c)){
                m.put(c,m.get(c)+1);
                }
            else {
            m.put(c,1);
            }
        }
        //System.out.print(m);
    for (char c : ran.toCharArray()){
        if(!m.containsKey(c)){
            return 0;
        }
        m.put(c,m.get(c)-1);
        }
    for(int val : m.values()){
        if (val<0){
            return 0; // checking over use of char
        }
    }
    return 1;
}
}
```

### fds23.java

```
/*
An e-commerce warehouse stores product batch codes as lowercase strings.

Due to supplier variations, some product codes are written in different character orders but actually
 represent the same product type (they contain exactly the same letters with the same frequency).

Your task is to group all product codes that represent the same product type together.

You may return the groups in any order.


Input Format
A list of strings, strs, representing product codes, separated by spaces

Output Format
A list of lists of strings, each sublist containing codes that are anagrams of each other.
Each group is printed in square brackets and codes separated by commas.
The order of groups and the order of codes within a group does not matter.

Each inner list contains product codes that represent the same product type.

 Constraints

1 <= strs.length <= 10^4

0 <= strs[i].length <= 100

Sample Test Cases

Sample Test Case 1

Input:
strs = eat tea tan ate nat bat

Output:
[[bat],[nat,tan],[ate,eat,tea]]

Sample Test Case 2

Input:
strs =

Output:
[[]]

Sample Test Case 3

Input:
strs = a

Output:
[[a]]
*/
```

```
/*import java.util.*;
class fds23{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        String sp [] = s1.split(" ");
        Set <List <String>> l1 = new HashSet<>(); //set of lists --> outer one

        for (int i = 0; i<sp.length;i++){
             Set <String > l2 = new HashSet<>();// clear on every pass so that i can reuse
             l2.add(sp[i]);
            for (int j = 0; j<sp.length;j++){
                if(sp[i].length()== sp[j].length()){
                    int res = checkana(sp[i],sp[j]);
                    if(res == 1){
                        l2.add(sp[j]);
                    }
                }
            }
            l1.add(new ArrayList<>(l2));
        }
        //System.out.print(l1);
        System.out.print(new ArrayList<>(l1));
    }
    public static int checkana(String s1,String s2){
        s1 = s1.toLowerCase();
        s2 = s2.toLowerCase();
        HashMap <Character , Integer>  m = new HashMap<>();
        for (char c: s1.toCharArray()){
            m.put(c,m.getOrDefault(c,0)+1);
        }
        for(char c : s2.toCharArray()){
            if(!m.containsKey(c)) return 0;
            m.put(c,m.get(c) -1);
        }
        for (int v : m.values()){
            if(v!=0) return 0;
        }
        return 1;
    }

}*/
// logic : all anagrams will have same sorted order
import java.util.*;
class fds23{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 [] = s.nextLine().split(" ");
        HashMap <String ,List<String >> m = new HashMap<>(); // to map string and list of strings
        for (int i = 0;i<s1.length;i++){
            char [] k = s1[i].toCharArray();
            Arrays.sort(k); // sorting using arrays -> see 18
            String sor = new String(k);
            if(m.containsKey(sor)){
                m.get(sor).add(s1[i]);
                m.put(sor,m.get(sor));
            }
            else{
                List <String> l = new ArrayList<>();
                l.add(s1[i]);
                m.put(sor,l);
            }
        }
        System.out.print(m);
    }}
```

### fds24.java

```
/*Given an array, return the k most frequent elements.
Input:
nums = [1,1,1,2,2,3]
k = 2
Output:
[1,2]

sample test case 1:
input=1 1 1 2 2 3
2
output=1 2
```

```
sample test case 1:
input=2 1 1 2 2 3 3 4
3
output=2 3 1*/
//my approch :
/*import java.util.*;
class fds24{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 [] = s.nextLine().split(",");
        Integer l [] = new Integer[s1.length];
        for (int i = 0;i<s1.length;i++){
            l[i] = Integer.parseInt(s1[i]);
        }
        int j = s.nextInt();

        HashMap<Integer,Integer> m = new HashMap<>();
        for (int k : l){
            m.put(k,m.getOrDefault(k,0)+1);
        }
        //System.out.print((m.values()).getClass().getName());
        List <Integer> l2 = new ArrayList<>(m.values());
        l2.sort(Collections.reverseOrder());
        List <Integer> l3 = l2.subList(0,j);
        List <Integer > res = new ArrayList<>();// top k values kada
        for (int v : l3){
           // System.out.print(v);
           for(int k : m.keySet()){
               if(m.get(k)==v) {res.add(k);}
           }
        }
        System.out.print(res);
    }
}*/
// ai one:
import java.util.*;
class fds24{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 [] = s.nextLine().split(" ");
        Integer l [] = new Integer[s1.length];
        for (int i = 0;i<s1.length;i++){
            l[i] = Integer.parseInt(s1[i]);
        }
        int j = s.nextInt();

        HashMap<Integer,Integer> m = new HashMap<>();
        for (int k : l){
            m.put(k,m.getOrDefault(k,0)+1);
        }
        PriorityQueue <Integer> pq = new PriorityQueue<>(
            (a,b) -> m.get(a) - m.get(b) // see in book
            );
        for (int i: m.keySet())    {
            pq.add(i); // automatic ga ascending order of lo pothadi
            if(pq.size()>j){
                pq.poll();     // top k kada , so pool chestunnam
            }
        }
        List <Integer> res = new ArrayList<>();
        while(!pq.isEmpty()){
            //System.out.print(pq.poll() + " ");
            res.add(pq.poll());

        }
        Collections.reverse(res);
        for (int i : res) System.out.print(i + " ");

    }
}
```

### fds26.java

```
/*Given a list of dates, sort them in ascending (chronological) order.

Date Format:
Each date is in the form:  dd mmm yyyy
•   dd  : 01 to 31
•   mmm : Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec
•   yyyy: four digit year
```

```
Input:
20 Oct 2052
06 Jun 1933
26 May 1960
20 Sep 1958
16 Mar 2068
25 May 1912
16 Dec 2018
26 Dec 2061
04 Nov 2030
28 Jul 1963

Output:
25 May 1912
06 Jun 1933
20 Sep 1958
26 May 1960
28 Jul 1963
16 Dec 2018
04 Nov 2030
20 Oct 2052
26 Dec 2061
16 Mar 2068

Concept: Custom Sort with key mapping (month name → month number)
*/
//shrewwwni approach :
import java.util.*;
class fds26
{
    public static void main(String args[])
    {
        Scanner s= new Scanner(System.in);
        int a=s.nextInt();
        s.nextLine(); // fix

        String q[]=new String[a];

        Map<String,Integer> mnth=new HashMap<>();
        mnth.put("Jan",1); mnth.put("Feb",2); mnth.put("Mar",3);
        mnth.put("Apr",4); mnth.put("May",5); mnth.put("Jun",6);
        mnth.put("Jul",7); mnth.put("Aug",8); mnth.put("Sep",9);
        mnth.put("Oct",10); mnth.put("Nov",11); mnth.put("Dec",12);

        for(int i=0;i<a;i++)
        {
            q[i]=s.nextLine();
        }

        for(int i=0;i<a-1;i++)
        {
            for(int j=0;j<a-i-1;j++)
            {
                String p[]=q[j].split(" ");
                String b[]=q[j+1].split(" ");

                int y1=Integer.parseInt(p[2]);
                int y2=Integer.parseInt(b[2]);

                if(y1 > y2 ||
                  (y1 == y2 && mnth.get(p[1]) > mnth.get(b[1])) ||
                  (y1 == y2 && mnth.get(p[1]).equals(mnth.get(b[1])) &&
                   Integer.parseInt(p[0]) > Integer.parseInt(b[0])))
                {
                    String temp=q[j];
                    q[j]=q[j+1];
                    q[j+1]=temp;
                }
            }
        }

        for(int i=0;i<a;i++)
        {
            System.out.println(q[i]);
        }
    }
}
```

### fds27.java

```
/*Playing with numbers is fun!!
Given an array of integers and a number k
, find the maximum sum among all contiguous subarrays whose sum is less than or equal to k.
 Return the maximum such sum possible.

Approach:
Use a Variable Sliding Window.
Expand right by adding elements to the current sum. When the sum exceeds k,
 shrink the window from the left. Track the maximum valid sum throughout the traversal.
Sample Input and Output 1:
arr = [1, 2, 3, 4, 5]
k   = 9
Output: 9
Explanation: [4,5] = 9 which is <= 9, and is the maximum possible

Sample Input and Output 2:
arr = [3, 1, 4, 1, 5, 9, 2, 6]
k   = 10
Output: 10
Explanation: [4,1,5] = 10 which is the maximum sum <= k

Sample Input and Output 3:
arr = [2, 4, 6, 8]
k   = 5
Output: 4
Explanation: Only single elements qualify; maximum is 4

Concept: Variable Sliding Window — expand right, shrink left when sum > k
 */
//jaya
import java.util.*;

class fds27 {
    public static void main(String args[]) {
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) {
            a[i] = s.nextInt();
        }
        int k = s.nextInt();
        int sum = 0, max = 0;
        int l= 0;
        for (int i=0;i< n;i++) {
            sum += a[i];
            while (sum > k && l<=i) {
                sum -= a[l];
                l++;
            }
            if (sum <= k) {
                max = Math.max(max, sum);
            }
        }
        System.out.println(max);
    }
}
// koncham manchiga
int sum = 0;
int res = Integer.MIN_VALUE;
int left = 0;
for (int right = 0;right<n;right++){
    sum += arr[right];
    while(sum >k) sum -= arr[left]; left --;
    res = Math.max(sum,res);
}
return res;
```

### fds28.java

```
// Programming includes many braces like '{..}', '[..]', '(..)'. Let's check for the correct nestedness of braces!

// NOTE: There are no spaces between the braces in the actual input. Spaces in examples are shown for ease of
understanding.

// Test Cases:
// Input  = {[()]}
// Output = TRUE

// Input  = {[(])}
// Output = FALSE
```

```
// Input  = {()[]}
// Output = TRUE

// Input  = {[()]}
// Output = TRUE

// Concept: Stack-based bracket matching

// import java.util.*;
//brinda
/*class fds28
{
    public static void main(String args[])
    {
        Scanner s = new Scanner(System.in);
        String str = s.nextLine();

        Stack<Character> stack = new Stack<>();

        for(char c : str.toCharArray())
        {
            // opening brackets
            if(c == '(' || c == '{' || c == '[')
            {
                stack.push(c);
            }
            else
            {


                char top = stack.pop();
                if((c == ')' && top != '(') ||
                   (c == '}' && top != '{') ||
                   (c == ']' && top != '['))
                {
                    System.out.println("FALSE");
                    return;
                }
            }
        }


        if(stack.isEmpty())
            System.out.println("TRUE");
        else
            System.out.println("FALSE");
    }
}*/
//chatgpt

import java.util.HashMap;
import java.util.Scanner;
import java.util.Stack;

class fds28{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        HashMap <Character> h = new HashMap<>();
        h.put('(',')');
        h.put('[',']');
        h.put('{','}');
        Stack <Character> stackuu  = new Stack<>();
        for(char c : s1.toCharArray()){
            if(h.containsKey(c)){ // anntey opening vi
                stackuu.push(c);
            }
            else if(h.containsValue(c)){
                char top = stackuu.pop();           // anntey closing
                if(h.get(top) != c){ // comparing using map
                    System.out.println("False");
                    return;
                }
            }
        }
        if(stackuu.isEmpty()){
            System.out.println("True");
            return;
        }
        else {
                System.out.println("False");
        }
```

```
    }
}
```

### fds32.java

```
class fds32{
    public static void main(String args[]){
        m1(1);
        System.out.println();
        m2(1);
        System.out.println();
        System.out.println(m3(3));
        System.out.println(m4(3));
        System.out.println(m5(1456)); // sum of digits
        m6(126);
        System.out.println(m7(15476,0));
       // System.out.println(m6(1456)); // rev of digits
    }
    public static void m1(int c){
        if(c == 5){
            return;
        }
        System.out.print(c + " ");
        m1(c+1);
    }
    public static void m2(int c){
        if(c == 5){
            return;
        }
        System.out.print(10 + " ");
        m2(c+1);
    }
    public static int m3(int c){
        if(c==1) return 1000;
        return (m3(c-1)+1000);
    }
    public static int m4(int c){
        if(c==0) return c;
        return (m4(c-1)+c);
    }
    public static int m5(int c){
        if(c==0) return c;
        return (m5(c/10)+c%10);

    }
    public static void m6(int c){ // cheating
        if(c==0) return;
        System.out.print(c%10);
        m6(c/10);

    }
    public static int m7(int c,int y){
        if (c==0) return y;
        int x = y*10 + (c%10);
        return m7(c/10,x);
    }

}
```

### fds5.java

```
/* remove dulpicatses from sorted array
*/
import java.util.*;
class fds5{
    public static void main(String args[]){
        int [] a = {1,2,2,3,3,4,5,6};
        Set <Integer> l = new LinkedHashSet<>(); // follows the input order
        for(int x : a){
            l.add(x);
        }
        System.out.print(l);

    }
}
```

### First_Unique_Character_in_a_String_387.java

```
/*
Imagine you are developing a customer complaint tracking system.
Each complaint is represented as a lowercase string S, where:
Each character represents a complaint category.
Some categories are repeated multiple times.
The system needs to identify the first complaint category that appears only once,
so it can be prioritized.
If no such unique complaint category exists, return -1.

Problem Statement
Given a string S consisting of only lowercase English letters,
 return the index of the first non-repeating character in the string.
If no unique character exists, return -1.
 Input Format:
A single string S
Output Format:
An integer representing the index of the first non-repeating character.
If none exists, return -1.
⚙ Constraints:
1 <= S.length <= 10^5
S consists only of lowercase English letters.
Sample Test Cases
Sample Input-1:
leetcode
Sample Output-1:
0
Sample Input-2:
loveleetcode
Sample Output-2:
2
Sample Input-3:
aabb
Sample Output-3:
-1
*/
/* idhi shreni and jaya code
import java.util.*;
class fds17

{


    public static void main(String args[])


{


        Scanner s= new Scanner(System.in);
        String m=s.nextLine().toLowerCase();
        Map<Character,Integer> an=new LinkedHashMap<>();
        for(char i:m.toCharArray())


{


            an.put(i,an.getOrDefault(i,0)+1);


}
       // System.out.println(an);
        boolean o=false;
        for(char i:an.keySet()){
            if(an.get(i)==1)
            {
                System.out.println(m.indexOf(i));
                o=true;
                return;


}


}
        if(o==false)


{
```

```
            System.out.println("-1");


}



}

}*/
//idhi na code
// and we can do by freq array aslo
import java.util.*;
public class First_Unique_Character_in_a_String_387({
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        //System.out.print(s1[2]);
        HashMap <Character, Integer> m = new HashMap<>();
        for(char c: s1.toCharArray()){
            m.put(c,m.getOrDefault(c,0)+1);
        }
        for(int i=0;i<s1.length();i++){
            if((m.get(s1.charAt(i)).equals(1))){
                System.out.print(i);
                return ;
            }
        }
        System.out.println("-1");
    }
}
/*int[] f = new int[26];
for (char c : s.toCharArray()) f[c - 'a']++;
for (int i = 0; i < s.length(); i++)
    if (f[s.charAt(i) - 'a'] == 1) return i;
return -1; */
//https://claude.ai/share/9ed98af4-9146-482d-833a-72436ce657b9
```

### lamda_func.java

```
public class lamda_func{
    public static void main(String[] args) {
        System.out.println(lambda ()-> "shashi");
    }
}
```

### left_rotation.java

```
/* left rotation by k steps */
// m - 1 :
import java.util.*;
class left_rotation{
    public static void reverse(int a[],int s,int e){
        while(s<e){
            int temp = a[s];
            a[s] =a[e];
            a[e] = temp;
            s++ ; e--;
        }
    }
    public static void main(String args[]){
        int a [] = {1,2,3,4,5,6};
        int k = 5;
        int n = a.length; // PAKKA length ee undali not n-1
        if(k>n) k = k%n;
        // s1 : revere complete
        reverse(a,0,n-1);
        System.out.print(Arrays.toString(a));
        int  h = n-k;
        reverse(a,0,h-1);
        System.out.print(Arrays.toString(a));
        //s3 : reverse remanining elements :
        reverse(a,h,n-1);
        System.out.print(Arrays.toString(a));
    }
}
// m - 2 : new array :
/*import java.util.*;

class RotateArray {
```

```
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int k = 2;

        int n = arr.length;
        int[] result = new int[n];

        for (int i = 0; i < n; i++) {
            result[i] = arr[(i+k)%n];
        }

        System.out.println(Arrays.toString(result));
    }
}
 */
//https://claude.ai/share/809403b9-17aa-4c58-b41e-a79c02f6579a
```

### Longest_Consecutive_Sequence_128.java

```
/*Given an array of integers, find the length of the longest
 subsequence such that elements in the subsequence are consecutive integers.
  The consecutive numbers can be in any order.

Sample Input and Output 1:
Input:  arr[] = {1, 9, 3, 10, 4, 20, 2}
Output: 4
Explanation: The subsequence 1, 2, 3, 4 is the longest consecutive subsequence

Sample Input and Output 2:
Input:  arr[] = {36, 41, 56, 35, 44, 33, 34, 92, 43, 32, 42}
Output: 5
Explanation: The subsequence 36, 35, 33, 34, 32 is the longest

Concept: Hashing / HashSet – O(n) approach
*/
import java.util.*;
public class Longest_Consecutive_Sequence_128{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 [] = s.nextLine().split(",");
        HashSet <Integer> h = new HashSet<>();
        ArrayList <Integer > l = new ArrayList<>();
        for (String i : s1){
            h.add(Integer.parseInt(i)); // ee step nen revison lo marchipoyina
        }
        //System.out.print(h);
        for (int i: h){
            if(!h.contains(i-1)){
                int c = 1;
                int j = i; // to go to next element
                while(h.contains(j+1)){
                    c++;
                    j++;
                }
                l.add(c); // to track all the counts
                //more correct : use math.max(c,res);
                }
            }
        l.sort(Collections.reverseOrder());
        System.out.print(l.get(0));
    }
    }
//approch 2:
/*public static int app2(int arr[]) {

        Arrays.sort(arr);

        int longest = 1;
        int c = 1;

        for(int i = 1; i < arr.length; i++) {

            if(arr[i] == arr[i-1]) {
                continue;
            }

            if(arr[i] == arr[i-1] + 1) {
                c++;
            }
            else {
                c = 1;
```

```
            }

            longest = Math.max(longest, c);
        }

        return arr.length == 0 ? 0 : longest; //idhi show kosam
    } */
```

### Longest_Substring_Without_Vowels.java

```
// ===========================================
// Given a string, find the length of the longest substring that contains no vowels (a, e, i, o, u).

// Approach:
// Use a Variable Sliding Window with two pointers left and right. Expand right freely across the string. When a
vowel is encountered, reset left to right + 1. Track the maximum window size throughout.

// Constraints:
// 1 <= length of string <= 50

// Sample Input and Output 1:
// Input:  ttrrcaamppttz
// Output: 4
// Explanation: 'pptt' is the longest window with no vowels (length 4)

// Sample Input and Output 2:
// Input:  aeiou
// Output: 0

// Sample Input and Output 3:
// Input:  rhythm
// Output: 6
// Explanation: Entire string has no vowels

// Concept: Variable Sliding Window — expand right, reset left on vowel hit
//ali bhai
import java.util.*;
public class Longest_Substring_Without_Vowels
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        String s=sc.nextLine().toLowerCase();
        int left=0,right=0;
        int maxLen=0;
        for(right=0;right<s.length();right++)
        {
            char ch=s.charAt(right);
            if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')
            {
                left=right+1; // reset cheayali
            }
            else
            {
                maxLen=Math.max(maxLen,right-left+1); // tracking max length
            }
            // else pettakapothey inka machidhi

        }
        System.out.println(maxLen);
    }
}
```

### Reverse_Array_In_Groups .java

```
/*Problem:
Reverse every group of k elements in an array. If remaining elements are less than k, reverse all of them.
Examples:
Input: arr = [1, 2, 3, 4, 5, 6, 7, 8], k = 3
Output: [3, 2, 1, 6, 5, 4, 8, 7]
Explanation: First 3 reversed, next 3 reversed, last 2 reversed

Input: arr = [1, 2, 3, 4, 5], k = 2
Output: [2, 1, 4, 3, 5]
What will happen for
k = 1
k >= array length
k = 0
*/
//my apporach :
```

```
/*import java.util.*;
class fds1{
    public static void main(String args[]){
        /*Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int k = s.nextInt();
        int a[] = new int[n];
        for(int i = 0;i<n;i++){
            a[i] = s.nextInt();
        }*/
    //    int a [] = {4,5,6,7,8,9,10,11};
    //    int k = 3;
       /*int l = 0;
        for(int i=0;i<a.length - 1;i=i+k-1){
            //System.out.println(i);
            int x = i;
            int j=i+k-1;
            l=j+1;
            //System.out.println(j);
            while(x<j){
                int temp = a[x];
                a[x] = a[j];
                a[j] = temp;
                x--;
                i++;

            }

        }
        //for last elements:
        int y = a.length - 1;
        while(l<y){
            int temp = a[l];
                a[l] = a[y];
                a[y] = temp;
                y--;
                l++;
        }

        for(int i = 0;i<a.length;i++){
            System.out.print(" "+a[i]);
        }
    }
}
*/
import java.util.*;
class Reverse_Array_In_Groups {
    public static void main(String args[]) {

        int[] a = {4, 5, 6, 7, 8, 9, 10, 11};
        int k = 3;
        int n = a.length;

        if (k <= 0) {
            System.out.println("Invalid k");
            return;
        }

        for (int i = 0; i < n; i += k) {

            int left = i;
            int right = Math.min(i + k - 1, n - 1); // heart of the problem

            while (left < right) {
                int temp = a[left];
                a[left] = a[right];
                a[right] = temp;
                left++;
                right--;
            }
        }

        System.out.println(Arrays.toString(a));
    }
}
/* shreni : idhi  ikkada panu cheyadu
public static void m1(int a1[],int a[],int k,int n){
        a1[0]=a[k];
        int i=k + 1;
        int j=1;
        while(i!=k){
            a1[j]=a[i];
            i=(i + 1)%n;
```

```
            j++;
        }
        System.out.print(Arrays.toString(a1));
    }*/
```

### Reverse_Words_in_a_String_151.java

```
/*Problem:
// same question with out split is there in strings ques 1 of interviews
Reverse the order of words in a string. Words are separated by spaces.
Examples:
Input: "the sky is blue"
Output: "blue is sky the"

Input: "  hello world  "
Output: "world hello"
(Note: Remove extra spaces)

Input: "a good   example"
Output: "example good a"
(Note: Reduce multiple spaces to single space)
Constraints:

Remove leading/trailing spaces
Reduce multiple spaces between words to single space*/
/*public class fds3 {
        public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String a = s.nextLine();
        String [] a2 = a.split("\\s+");
        int i=0, j = a2.length- 1;
        // two pointers vadudam :
        while(i<j){
            String temp = a2[i];
            a2[i] = a2[j];
            a2[j] = temp;
            i++;
            j--;
        }
        System.out.print(Arrays.toString(a2));
    }
}
*/
import java.util.*;

public class Reverse_Words_in_a_String_151{
    public static void main(String args[]) {
        Scanner s = new Scanner(System.in);
        String a = s.nextLine();
        String[] a2 = a.trim().split("\\s+");   // trim first
        int i = 0, j = a2.length - 1;
        // two pointers vadudam :
        while (i < j) {
            String temp = a2[i];
            a2[i] = a2[j];
            a2[j] = temp;
            i++;
            j--;
        }
        System.out.print(String.join(" ", a2)); // join with single space
    }
}
```

### Reverse_Words_in_a_String_III.java

```
/*Problem:
Reverse characters in each word but maintain word order.
Examples:
Input: "Let's take LeetCode contest"
Output: "s'teL ekat edoCteeL tsetnoc"

Input: "Hello World"
Output: "olleH dlroW"*/
import java.util.*;
class Reverse_Words_in_a_String_III{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String a = s.nextLine();
        String [] a2 = a.split(" ");
```

```
        StringBuilder res  = new StringBuilder();
        for(String x : a2){
            char [] x2 = x.toCharArray();
            int i=0,j=x.length() - 1;
            while(i<j){
                char temp = x2[i];
                x2[i] = x2[j];
                x2[j] = temp;
                i++; j--;

            }
            // System.out.println(x2);
            res.append(x2);
            res.append(" ");
        }
        System.out.print((res));
    }
}
```

### right_rotation_189.java

```
/* right rotation by k steps */
// m - 1 :
import java.util.*;
public class right_rotation_189{
    public static void reverse(int a[],int s,int e){
        while(s<e){
            int temp = a[s];
            a[s] =a[e];
            a[e] = temp;
            s++ ; e--;
        }
    }
    public static void main(String args[]){
        int a [] = {1,2,3,4,5,6};
        int k = 2;
        // super imp : if k > n then use k = k % n
        int n = a.length;
        // s1 : revere complete
        reverse(a,0,n-1);
        System.out.print(Arrays.toString(a));
        //s2: reverse upto k elements
        reverse(a,0,k-1);
        System.out.print(Arrays.toString(a));
        //s3 : reverse remanining elements :
        reverse(a,k,n-1);
        System.out.print(Arrays.toString(a));
    }
}
// m - 2 : new array :
/*import java.util.*;

class RotateArray {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int k = 2;

        int n = arr.length;
        int[] result = new int[n];

        for (int i = 0; i < n; i++) {
            result[(i + k) % n] = arr[i];
        }

        System.out.println(Arrays.toString(result));
    }
}
 */
```

### Rotate_Array_189_3.java

```
/*Problem Statement:
You're building a playlist shuffle feature for a music app. The app has a "rotate playlist" button that shifts all
songs by k positions to give users a different listening order without full randomization.
Given an array of songs and a number k, rotate the array to the right by k steps.

Example Scenarios:

Input: songs = ["Pop", "Rock", "Jazz", "Blues", "Country"], k = 2
Output: ["Blues", "Country", "Pop", "Rock", "Jazz"]
```

```
Explanation:
After 1st rotation: ["Country", "Pop", "Rock", "Jazz", "Blues"]
After 2nd rotation: ["Blues", "Country", "Pop", "Rock", "Jazz"]

Input: songs = ["Song1", "Song2", "Song3"], k = 1
Output: ["Song3", "Song1", "Song2"]

Explanation: Move the last song to the front

Input: nums = [1, 2, 3, 4, 5], k = 5
Output: [1, 2, 3, 4, 5]

Explanation: Rotating by array length brings it back to original


*/
import java.util.*;

public class Rotate_Array_189_3 {
    public static void main(String[] args) {

        String[] songs = {"Pop", "Rock", "Jazz", "Blues", "Country"};
        int k = 2;

        int n = songs.length;
        k = k % n;   // handle large k

        String[] result = new String[n];

        for (int i = 0; i < n; i++) {
            result[(i + k) % n] = songs[i];
        }

        System.out.println(Arrays.toString(result));
    }
}
```

### rotate_array_796.java

```
/*Problem Statement:
Given two strings s and goal, return true if goal is a rotation of s, otherwise return false.

Example 1:
Input: s = "abcde", goal = "cdeab"
Output: true

Example 2:
Input: s = "abcde", goal = "abced"
Output: false

Example 3:
Input: s = "aa", goal = "aa"
Output: true



Constraints:

1 <= s.length, goal.length <= 100
s and goal consist of lowercase English letters */
import java.util.*;
class rotate_array_796{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String k = s.next();
        k = k.toLowerCase();
        String t = s.next();
        t = t.toLowerCase();
        if (k.length() != t.length()){ // rotate cchhesina anntha matranna length maradu
            System.out.print("false");
            return;
}
        String l = k + k;
        if(l.contains(t)){
            System.out.print("true");
        }
        else{
            System.out.print("false");
        }
```

```
    }
}
// koncham smart ga rayali anntey :(claude)
/*public boolean rotateString(String s, String goal) {
    return s.length() == goal.length() && (s + s).contains(goal);
}*/
//https://claude.ai/share/6fc7a698-fa55-49d7-b0a6-84d41edd9b61
```

### rotate_array_796_2.java

```
/*String Rotation - Scenario Question

Problem: Security Password Rotation System
You're building a security system where passwords are rotated daily for extra protection.
The system takes the original password and shifts some characters from the beginning to the end each day.
Your manager asks you to:

Verify if today's password is a valid rotation of yesterday's password
Report how many positions it was shifted (for audit logs)

If there's been a security breach (password changed incorrectly), report it.
Write a function to check if the new password is a valid rotation and return the shift amount.

Example 1: Valid Daily Rotation
Input:
yesterday_password = "abcde"
today_password = "cdeab"

Output: (true, 2)

Explanation:
The system moved "ab" (2 characters) from start to end.
Valid rotation - shifted by 2 positions!

Example 2: Security Breach Detected!
Input:
yesterday_password = "abcde"
today_password = "abced"

Output: (false, -1)

Explanation:
Letters are in wrong order - NOT a rotation!
Return -1 to indicate breach.

Example 3: No Rotation (Same Password)
Input:
yesterday_password = "aa"
today_password = "aa"

Output: (true, 0)

Explanation:
Password unchanged (or rotated by full length).
Shifted by 0 positions.


Constraints:

Password length: 1 to 100 characters
Passwords contain only lowercase English letters
Both passwords must be same length to be valid rotations */
import java.util.*;
class rotate_array_796_2

{


    public static void main(String args[])

{


        Scanner s = new Scanner(System.in);
        String k = s.next();
        k = k.toLowerCase();
        String t = s.next();
        t = t.toLowerCase();
        if (k.length() != t.length())

{
```

```
 // rotate cchhesina anntha matranna length maradu
            System.out.print("(" + "false" + "," + "-1" + ")");
            return;

}
        String l = k + k;
        if(l.contains(t))

{


            System.out.print("(" + "true" + "," + l.indexOf(t) + ")");


}
        else

{


           System.out.print("(" + "false" + "," + l.indexOf(t) + ")");


}


}

}
```

### streams.java

```
//streams : it is a like pipe line which have 3 func -> source(stream()) , intermidiate(filter,map,find any,match
any),stopping (collector,count,for each)
import java.util.*;
import java.util.function.Predicate; //predicates kosam
public class streams{
    public static void main(String args[] ){
        List<Integer> l = new ArrayList<>(Arrays.asList(10,20,30,40,50));
        List<String> l2 = new ArrayList<>(Arrays.asList("anil","sarath","zoe","mandira","akshara","anand"));
        List<Integer> scores = Arrays.asList(45, 82, 61, 38, 75, 90, 55, 48, 70, 33);
        //System.out.print(l);
        Predicate <Integer> p = (n) -> n>40;
        Predicate <Integer> p2 = (n) -> n%3==0;
        Predicate <Integer> k = (n) -> n>60;
        Predicate <Integer> k2 = (n) -> n%2==0;
        Predicate <Integer> k3 = (n) -> n>50;
        Predicate <Integer> k4 = (n) -> n<50;
        Predicate <Integer> ks = (n) -> n>70;
        Predicate <String> s = (n) ->n.startsWith("a");
        Predicate <String> s2 = (n) ->n.length()<=5;
        l.stream().filter(p.or(p2)).forEach(n-> System.out.println(n));
        l2.stream().filter(s).forEach(n-> System.out.print(n+" "));
        l2.stream().filter(s2).forEach(n-> System.out.println(n));
        l2.stream().map((n) ->n.toUpperCase()).forEach(n-> System.out.print(n+ " "));
        scores.stream().filter(k).forEach(n-> System.out.print(n + " "));
        System.out.println();
        scores.stream().filter(k2.and(k3)).forEach(n-> System.out.print(n + " "));

        System.out.println();
        scores.stream().filter(k4).forEach(n-> System.out.print(n + " "));

        System.out.println();


    }
}
```

### Valid_Anagram_242.java

```
//anagram
/*A cybersecurity system needs to verify whether two encrypted messages contain exactly the same
characters in the same frequency but possibly in different orders.
You are given two strings s and t.
Your task is to determine whether string t is an anagram of string s.
Two strings are said to be anagrams if:
They contain the same characters.
Each character appears the same number of times
The order of characters does not matter.
```

```
Return true if t is an anagram of s, otherwise return **false`.

Input Format:
First line: String s
Second line: String t

Output Format
Print true if t is an anagram of s
Print false otherwise
Output must be in lowercase.

Constraints
1 ≤ s.length ≤ 5 × 10^4
1 ≤ t.length ≤ 5 × 10^4
s and t consist only of lowercase English letters.


sample Test Case 1:
input=listen
silent
output=true

Sample Test Case 2:
input=hello
bello
output=false
*/

import java.util.*;

public class Valid_Anagram_242{
    public static void anacheck(String s1,String t){
        s1 = s1.toLowerCase();
        t = t.toLowerCase();
        if(s1.length()!= t.length()){ // ofc
            System.out.print(false);
            return;
        }
        Map <Character,Integer> m = new HashMap<>();
        //System.out.println(s1);
        for (char c : s1.toCharArray()){
            m.put(c,m.getOrDefault(c,0) + 1); // to find freq
        }
        System.out.println(m);
        for (char c : t.toCharArray()){
            if(!m.containsKey(c)){
                System.out.println(false);
                return ; // kotha alaphabet ravaddu kada
            }
            m.put(c,m.get(c)-1); // zero down cheddam ani idea kada
        }
        System.out.println(m);
        for (int v : m.values()){
            if(v!=0){
                System.out.println(false); // freq different ayithe zero radu
                return ;
            }
        }
        System.out.print(true);
    }
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        String t = s.nextLine();
        anacheck(s1,t);
    }
}
// optimal : telsu ga freq array unntadi ani (heart : c - 'a' gives value in int )
/*if (s.length() != t.length()) { System.out.println(false); return; }
int[] cnt = new int[26];
for (int i = 0; i < s.length(); i++) {
    cnt[s.charAt(i) - 'a']++;
    cnt[t.charAt(i) - 'a']--;
}
for (int c : cnt) if (c != 0) { System.out.println(false); return; }
System.out.println(true);*/

//brute:
/*char[] a = s.toCharArray(), b = t.toCharArray();
Arrays.sort(a); Arrays.sort(b);
System.out.println(Arrays.equals(a, b)); */
```

### Valid_Anagram_Application_242.java

```
/*
A warehouse dispatch system recorded the outgoing shipment as a string s.
Each character in the string represents a type of item (lowercase English letters).
After delivery, the received shipment is recorded as string t.

It is observed that:
t contains all original items from s (possibly shuffled).
Exactly one extra item was mistakenly added during shipment.
Your task is to identify and return the extra item type.

 Input Format
s = original_shipment
t = received_shipment

Output Format
extra_item

Return the single lowercase character that represents the extra item.
 Constraints
0 <= s.length <= 1000
t.length == s.length + 1
s and t consist only of lowercase English letters

Exactly one extra item exists in t

Sample Test Cases
Test Case1
input=programming
rprogramming
output=r
Test Case2
input=zzz
zzzz
output=z

*/
/*
//idhi ali bhai approach
// import java.util.*;
class fds15
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String a = sc.nextLine();
        String b = sc.nextLine();
        HashMap<Character, Integer> a1 = new HashMap<>();
        HashMap<Character, Integer> b1 = new HashMap<>();
        for(char i: a.toCharArray())
        {
            a1.put(i, a1.getOrDefault(i, 0)+1);
        }
        for(char j: b.toCharArray())
        {
            b1.put(j, b1.getOrDefault(j, 0)+1);
        }
        for(char k: b1.keySet())
        {
            int c1 = a1.getOrDefault(k, 0);
            int c2 = b1.get(k);
            if(c1<c2)
            {
                System.out.println(k);
            }
        }
    }
}*/
// lets do by freq array
// idhi nenu chesina
import java.util.*;
public class Valid_Anagram_Application_242{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        String s2 = s.nextLine();
        int freq [] = new int [26];
        for (char c : s1.toCharArray()){
            freq[c - 'a']++;
        }
        for(char c : s2.toCharArray()) freq[c-'a']--;
        for (int i=0;i<freq.length;i++){
```

```
            if(freq[i]!=0){
            System.out.println((char)(i+97)); // wow noiceee
            return ;

            }
        }
    }
}
```

### Word_Pattern_290.java

```
/*
/*
A company uses a badge coding system to assign employee access roles.

A string pattern represents the badge role sequence (each character = role type).

A string s represents actual employee names assigned in sequence (space-separated).

Your task is to determine whether the badge assignment follows a strict one-to-one mapping rule:

Rules

Each role character in pattern must map to exactly one unique employee name.

Each employee name must map to exactly one role character.

No two different roles can map to the same employee.

No two different employees can map to the same role.

Return true if the assignment is valid. Otherwise, return false.

Input Format
pattern = ""<pattern_string>""
s = ""<space_separated_employee_names>""
Output Format
true or false
Constraints

1 <= pattern.length <= 300

pattern contains only lowercase English letters

1 <= s.length <= 3000

s contains lowercase English letters and single spaces

No leading or trailing spaces

Words are separated by exactly one space

Sample Test Cases

Test Case 1
input=pattern = abba
s = dog cat cat dog

output=true

Explanation:
Mapping:
a → dog
b → cat

Test Case 2

input=pattern = abba
s = dog cat cat fish

output=false

Explanation:
'b' cannot map to both 'cat' and 'fish'

*/
import java.util.*;
public class Word_Pattern_290{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
```

```
        String sk = s.nextLine();
        //int res = twomaps_approach(s1,sk);
        int res = oneset_onemap_apporach(s1,sk);
        System.out.print(res == 1); // boolean lo vastadi
    }
    public static int twomaps_approach(String s1,String sk){
        String s2 [] = sk.split(" ");
        int i = 0;
        Map <Character,String > m1 = new HashMap<>();
        Map <String , Character> m2 = new HashMap<>();
        for (char ch : s1.toCharArray()){
            String word = s2[i];
            if(!(m1.containsKey(ch)) && !(m2.containsKey(word))){ // both were not mapped eariler kada
            m1.put(ch,word); m2.put(word,ch);
            }
            else if(!m1.containsKey(ch) || !m2.containsKey(word)){ // very imp case see book
                return 0;
            }
            else { // annthey both contain
                if(!m1.get(ch).equals(word) || !m2.get(word).equals(ch)) // best use of or gate --> see book
                return 0;
            }
            i++;
        }
        return 1; //ikkadi varaku vaccham anntey no problem ani artham
    }
    public static int oneset_onemap_apporach(String s1,String sk){
        String s2 [] = sk.split(" ");
        if (s1.length()!=s2.length) return 0; // enduko see in book
        Map <Character , String > m = new HashMap<>();
        Set <String > mapped = new HashSet<>(); // to keep a track of who are mapped till now
        int i = 0;
        for (char c : s1.toCharArray()){
            String word = s2[i];
            if(m.containsKey(c)){
                if(!(m.get(c).equals(word))){
                    return 0;
                }
            }
            else { // enters if key is not present
                //case 1 emo word is not already used
                if(mapped.contains(word)){
                    return 0;
                }
                else { // idi emo both are not present then simply add
                    m.put(c,word); mapped.add(word);
                }
            }
            i++;
        }
        return 1;
    }
}
```

### Word_Pattern_290_1.java

```
/*
Given two strings s and t, determine if they are isomorphic.
Two strings are isomorphic if characters in s can be replaced to get t, with a one-to-one mapping.
Input:
s = egg
t = add
Output:
true
Sample Test case1
input=egg
add
output=true

Sample Test case2
input=foo
bar
output=false

*/
//i assumed it was only way mapping but kadu annta
/*import java.util.*;
class fds31{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
```

```
        String s2 = s.nextLine();
        int i = 0; // to track elements of 2nd array
        // ig only one mapping ee anukunnta so only one map use cheedam
        HashMap <Character,Character> h = new HashMap<>();
        for (char c : s1.toCharArray()){
            if(h.containsKey(c)){ // anntey already eppudo map ayyindi
                if(!(h.get(c).equals(s2.charAt(i)))) {
                    System.out.print("False");
                    return;
                }
            }
            else{
                h.put(c,s2.charAt(i));
                i++;
            }
        }
    System.out.print("Ture"); // last ki vachesam kada
    }
}*/
/*import java.util.*;
class fds31{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        String s2 = s.nextLine();
        if(s1.length() != s2.length()){
            System.out.print("False");
            return ;
        }
        int i = 0; // to track elements of 2nd array
        // ig only one mapping ee anukunnta so only one map use cheedam
        HashMap <Character,Character> h = new HashMap<>();
        for (char c : s1.toCharArray()){
            if(h.containsKey(c)){ // anntey already eppudo map ayyindi
                if(!(h.get(c).equals(s2.charAt(i)))) {
                    System.out.print("False");
                    return;
                }
            }
            else{
                h.put(c,s2.charAt(i));
               // i++; logic : ikkada increment cheste tappu , see ai
            }
            i++;
        }
    System.out.print("True"); // last ki vachesam kada
    }
}*/
// this is code for 2 way mapping , denni inka optimise chesa in fds 18
/*import java.util.*;
class fds31

{


    public static void main(String args[])

{


        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        String s2 = s.nextLine();
        if(s1.length() != s2.length())

{


            System.out.print("False");
            return;


}
        int i = 0;
        // bijection kada so two maps vadali
        HashMap <Character , Character> m1 = new HashMap<>();
        HashMap <Character , Character> m2 = new HashMap<>();
        for (char c : s1.toCharArray())

{


            if(m1.containsKey(c))
```

```
{


                if(!(m1.get(c).equals(s2.charAt(i))))

{


                    System.out.print("False");
                    return;


}


}
            else

{


                m1.put(c,s2.charAt(i));


}
            i++;


}
        i = 0;
        for (char c : s2.toCharArray())

{


            if(m2.containsKey(c))

{


                if(!(m2.get(c).equals(s1.charAt(i))))

{


                    System.out.print("False");
                    return;


}


}
            else

{


                m2.put(c,s1.charAt(i));


}
            i++;


}
        System.out.print("True");


}

}*/
```

### Word_Pattern_290_2.java

```
/*/*
A data migration system is transferring records from Server S to Server T.

Each record in Server S is encoded using certain symbols.
During migration:
```

```
Every symbol from Server S must consistently translate to exactly one symbol in Server T.

No two different symbols from Server S can translate to the same symbol in Server T.

A symbol may translate to itself.

The order of symbols must remain unchanged.

Your task is to determine whether the encoding transformation from Server S to Server T is valid.

Return true if the migration rule is valid, otherwise return false.

 Input Format
s
t

s → Original encoded string from Server S

t → Transformed string from Server T

Output Format
true / false

Constraints

1 <= s.length <= 5 * 10^4

t.length == s.length

s and t consist of any valid ASCII characters

Order must be preserved

Mapping must be one-to-one (bijection)

Sample Test Cases
Test case 1
input =
egg
add

output =
true

Test case 2
input =
f11
b23

output =
false

*/
//ali bhai apprach
import java.util.*;

public class Word_Pattern_290_2{
    public static boolean A(String s, String t) {
        if (s.length() != t.length()) return false;

        HashMap<Character, Character> mapST = new HashMap<>();
        HashMap<Character, Character> mapTS = new HashMap<>();

        for (int i = 0; i < s.length(); i++) {
            char cs = s.charAt(i);
            char ct = t.charAt(i);
            if (mapST.containsKey(cs)){
                if (mapST.get(cs) != ct){
                    return false;
}

}
    else{
                mapST.put(cs, ct);
}

            if (mapTS.containsKey(ct)){
                if (mapTS.get(ct) != cs){
                    return false;
}

}
```

```
          else{
                mapTS.put(ct, cs);
          }

}

        return true;
}
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        String t = sc.nextLine();
        boolean r = A(s, t);
        if (r) {
            System.out.println("True");
}
        else{
            System.out.println("False");
         }
}

}
```

### zeroes_to_end.java

```
/*### **Problem: Movie Theater Seat Arrangement**

You're managing a movie theater's seating system.
After cancellations, some seats become empty (represented by 0).
You need to move all empty seats to the end while keeping the order of occupied seats the same.

- **0** = Empty seat
- **Non-zero numbers** = Ticket numbers of occupied seats

**Move all empty seats (zeros) to the end while maintaining the relative order of occupied seats.**

---

### **Example 1:**
```
Input: seats = [12, 0, 5, 0, 3, 0, 8]
Output: [12, 5, 3, 8, 0, 0, 0]

Explanation:
Occupied seats: 12, 5, 3, 8 (order maintained)
Empty seats: 0, 0, 0 (moved to end)
```

---

### **Example 2:**
```
Input: seats = [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]

Explanation:
Occupied seats stay in order: 1, 3, 12
Empty seats moved to end: 0, 0
```

---


---

### **Constraints:**
- 1 <= seats.length <= 10000
- 0 <= seats[i] <= 10000
- Must modify the array **in-place** (no extra array allowed)
- Maintain the relative order of non-zero elements

---

### **Your Task:**

Implement: `rearrange_seats(seats)`

The function should modify the input array directly.

---
*/
```

```
// u made me yappy

/*import java.util.*;
class sol{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int a[] = new int[n];
        int l=0,p=n-1;
        for(int i = 0;i<n;i++){
            int k = s.nextInt();
            if(k!=0){
                a[l] = k;
                l++;
            }
            else{
                a[p] = k;
                p--;
            }

        }
        System.out.println(Arrays.toString(a));
    }
}*/
//using two pointer approach
import java.util.*;
class Zeroes_to_end{

    public static void main(String aragS[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = s.nextInt();
        }
        int i=0;
        // 1st zero
        while(i<n && arr[i]!=0){
            i++;
        }
        int j = i+1; // idi 1st declare cheayddu
        while(j<n){
            if(arr[j]!=0){
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j]=temp;
                i++; // 1st zero position munduku potunndi kada (i eppudu 1st zero ke point chestunndi )
            }
            j++;
            // i only swap ayithane increment, kani j ofc pratisari increment kavali kada
            System.out.print(Arrays.toString(arr));
        }
    }
}
/* idhi inko approach :
class Solution {
    public void moveZeroes(int[] nums) {
        // overwrite all non zeros
        int k=0,i=0;
        while(i<nums.length){
            if(nums[i]!=0){
                nums[k++] = nums[i];
            }
            i++;
        }
        while(k<nums.length){
            nums[k++]=0;
        }
    }
        // non zero find out chesi , positon lo insert chesi increment chesi wait chestunnam (method explanation)

}` */
```

# may_holidays / arrays

### ques1.java

```
/*INTERVIEW QUESTION:

You are given an integer array of size n.
Find both the maximum and minimum elements in the array.

CONSTRAINTS:
- Solve it in a single traversal
- Do NOT use sorting
- Array can contain negative numbers
- n >= 1

EXAMPLE:
Input:  [3, 5, 1, 8, 2]
Output: min = 1, max = 8

FOLLOW-UP QUESTIONS:
1. Can you reduce the number of comparisons?
2. What is the minimum number of comparisons required?
3. Can you solve it using pair-wise comparison optimization?

YOUR TASK:
1. Explain your approach
2. Give time complexity
3. Write Java code*/
//my approach

/*import java.util.Scanner;

public class ques1 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String [] s1 = s.nextLine().split(" ");
        Integer k [] = new Integer[s1.length];
        for (int i=0;i<s1.length;i++){
            k[i] = Integer.parseInt(s1[i]);
        }
        int max = k[0];
        int min = k[0];
                //single traversal lo cheayli kabbatti :
        for(int i=0;i<k.length;i++){
            max = Math.max(max,k[i]);
            min = Math.min(min,k[i]);
        }
        System.out.printf("max is %d", max);
        System.out.printf("min is %d", min);

    }
}*/
// ai approach
import java.util.*;

public class ques1 {
    public static void main(String args[]) {
        Scanner s = new Scanner(System.in);
        String[] input = s.nextLine().split(" ");

        int[] arr = new int[input.length];
        for (int i = 0; i < input.length; i++) {
            arr[i] = Integer.parseInt(input[i]);
        }

        int min, max;
        int i;

        // Step 1: Initialize
        if (arr.length % 2 == 0) {
            // if even
            if (arr[0] < arr[1]) {
                min = arr[0];
                max = arr[1];
            } else {
                min = arr[1];
                max = arr[0];
            }
            i = 2;
        } else {
            // if odd
            min = max = arr[0];
```

```
            i = 1;
        }

        // Step 2: Process in pairs
        while (i < arr.length - 1) {
            int localMin, localMax;

            if (arr[i] < arr[i + 1]) {
                localMin = arr[i];
                localMax = arr[i + 1];
            } else {
                localMin = arr[i + 1];
                localMax = arr[i];
            }

            if (localMin < min) {
                min = localMin;
            }

            if (localMax > max) {
                max = localMax;
            }


            i += 2;
        }

        System.out.println("min = " + min + ", max = " + max);
    }
}
```

### ques10.java

```
import java.util.*;
public class ques10 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int m = s.nextInt();
        int n = s.nextInt();
        int arr[][] = new int [m][n];
        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++){
                arr[i][j]= s.nextInt();
            }
        }
        app2(arr,m,n);
        Set <Integer> r = new HashSet<>();
        Set <Integer > c = new HashSet<>();
        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++){
                if(arr[i][j]==0){
                    r.add(i);
                    c.add(j);
                }
            }
        }
       //System.out.println(r);
       for(int x:r){
        for(int i=0;i<n;i++){
            arr[x][i]=0;
        }
       }
       for(int x:c){
        for(int i=0;i<m;i++){
            arr[i][x]=0;
        }
       }
       // print
       for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
            System.out.print(arr[i][j]);
        }
        System.out.print("\n");
       }
}
public static void app2(int arr[][],int m,int n){
    boolean row0is0 = false;
    boolean col0is0 = false;
    for(int i=0;i<n;i++){
        if(arr[0][i]==0){
            row0is0 = true;
```

```
            break;
        }
    }
    for(int i=0;i<m;i++){
        if(arr[i][0]==0){
            col0is0 = true;
            break;
        }
    }
    for(int i=1;i<n;i++){
        for(int j=0;j<n;j++){
            if(arr[i][j]==0){
                arr[0][j] = 0;
                arr[i][0] = 0;
            }
        }
    }
    for(int i=0;i<m;i++){
        if(arr[i][0]==0){
            for(int j=1;j<n;j++){
                arr[i][j]=0;  // zero down
            }
        }
    }
    for(int i=0;i<n;i++){
        if(arr[0][i]==0){
            for(int j=1;j<n;j++){
                arr[j][i]=0;  // zero down
            }
        }
    }
    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
            System.out.print(arr[i][j]);

        }
        System.out.print("\n");
    }
}
}
```

### ques2.java

```
/*
INTERVIEW QUESTION:

You are given an integer array of size n.
Find the SECOND LARGEST element in the array.

CONSTRAINTS:
- Solve it in a single traversal
- Do NOT use sorting
- Handle duplicates properly
- If second largest does not exist, return -1

EXAMPLES:
Input:  [3, 5, 1, 8, 2]
Output: 5

Input:  [10, 10, 10]
Output: -1

Input:  [7]
Output: -1

FOLLOW-UP QUESTIONS:
1. What if the array contains negative numbers?
2. Can you do it in one pass?
3. What edge cases should you handle?

YOUR TASK:
1. Explain your approach
2. Give time complexity
3. Write Java code
*/
//epoch 1
/*import java.util.*;
public class ques2{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String [] s1 = s.nextLine().split(",");
```

```
        int k [] = new int[s1.length];
        for(int i=0;i<s1.length;i++){
            k[i] = Integer.parseInt(s1[i]);
        }
        int max_1 = k[0];
        int max_2 = k[0];
        for(int i=0;i<k.length;i++){
            max_2 = max_1;
            max_1 = Math.max(k[i],max_1);
        }
        System.out.print(max_2);


    }
}*/
/*//epoch 2
import java.util.*;
public class ques2{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String [] s1 = s.nextLine().split(",");
        int k [] = new int[s1.length];
        for(int i=0;i<s1.length;i++){
            k[i] = Integer.parseInt(s1[i]);
        }
        int max_1 = k[0];
        int max_2 = k[0];
        for (int i=0;i<k.length;i++){
            if (k[i]>max_1){
                max_2 = max_1;
                max_1 = k[i];
            }
            else if(k[i]>max_2){
                max_2 = k[i];
            }

        }
        System.out.println(max_2);
    }}*/
//epoch 3
import java.util.*;
public class ques2{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String [] s1 = s.nextLine().split(",");
        int k [] = new int[s1.length];
        for(int i=0;i<s1.length;i++){
            k[i] = Integer.parseInt(s1[i]);
        }
        int max_1 = Integer.MIN_VALUE;
        int max_2= Integer.MIN_VALUE;
        for (int i=0;i<k.length;i++){
            if (k[i]>max_1){
                max_2 = max_1;
                max_1 = k[i];
            }
            else if(k[i]>max_2 && k[i]!=max_1){
                max_2 = k[i];
            }

        }
        if(max_2==Integer.MIN_VALUE){
            System.out.print("-1");
            return;
        }
        System.out.println(max_2);
    }}
```

### ques3.java

```
/*t.Given an array of integers, return the indices of the two numbers that add up to a target.
Input:  nums = [2, 7, 11, 15], target = 9
Output: [0, 1]  // because nums[0]+nums[1] = 9
Constraints: Exactly one solution exists. Cannot use same element twice.
Follow-up: What if you need all pairs, not just indices? What if the array is sorted? */
import java.util.*;

public class ques3{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
```

```
        int arr [] = new int[n];
        for(int i =0;i<n;i++){
            arr[i] = s.nextInt();

        }
        System.out.println("target: ");
        int k = s.nextInt();
        appraoch_1(arr,k); // brute force
        approach_2(arr,k); // two pointer
        approach_3(arr, k);
    }
    public static void appraoch_1(int arr[],int k){
        for(int i=0;i<arr.length;i++){
            for(int j=i+1;j<arr.length;j++){
                if(arr[i] + arr[j] == k){
                    System.out.println(i+","+j);
                    return ;
                }
            }
        }
    }
    public static void approach_2(int arr[], int k){

        int left = 0 , right = arr.length-1;
        while (left < right){
            if (arr[left] + arr[right] == k){
                System.out.print(left+","+right);
                return;
            }
            else if((arr[left] + arr[right]>k)){
                right--;
            }
            else{
                left++;
            }
        }
    }
    public static void approach_3(int arr[], int k){
        Map<Integer,Integer> m = new HashMap<>();
        for(int i=0;i<arr.length;i++){
            if(m.containsKey(k-arr[i])){
                System.out.print(m.get(k-arr[i]) + " ," + i);
                return ;

            }
            else {
                m.put(arr[i],i);
            }
            System.out.println(i + " :" + m);
        }


    }
}
```

### ques4.java

```
/*Given a sorted array, remove duplicates in-place such that each unique element
 appears only once. Return the new length.
Input:  [1, 1, 2, 3, 3, 4]
Output: 4  // array is now [1, 2, 3, 4, ...]
Constraints: O(1) extra space. Modify array in-place.
Hint
Hide follow-up
Mark solved
Explain solution ↗
Follow-up: What if each element is allowed to appear at most twice? */
import java.util.*;
public class ques4{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 [] = s.nextLine().split(",")        ;
        int arr [] = new int [s1.length];
        for (int i=0;i<s1.length;i++){
            arr[i] = Integer.parseInt(s1[i]);
        }
        int i=0, j = 1;
        while(j<s1.length){
            if(arr[i]== arr[j]){
                j++;
            }
```

```
            else{
                arr[i+1] = arr[j]; //overwrite cheayli anthey
                i++; j++;
                System.out.println("Arrat now :"+ Arrays.toString(arr));

            }
        }
        System.out.println("Array length: "+ i+1);

    }
}
```

### ques5.java

```
// follow up of previous ques
import java.util.*;
class ques5{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 [] = s.nextLine().split(",");
        int arr [] = new int [s1.length];
        for(int i=0;i<s1.length;i++){
            arr[i] = Integer.parseInt(s1[i]);

        }
        int i=0, j=1 , count = 0;
        while(j<s1.length){
            if(arr[i] == arr[j]){

            }
        }
    }
}
```

### ques6.java

```
/*Implement a phone number letter combination generator. Given a digit string, return all possible letter
combinations (like a T9 keyboard).
Input:  "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
Constraints: Digits 2-9 only. Use a Map for digit-to-letters mapping.
Hint
Hide follow-up
Mark solved
Explain solution ↗
Follow-up: How many combinations are possible for a 10-digit number? Is recursion or iteration better here?*/
import java.util.*;
public class ques6 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        HashMap <Character,String > map = new HashMap<>();
        map.put('2', "abc");
        map.put('3', "def");
        map.put('4', "ghi");
        map.put('5', "jkl");
        map.put('6', "mno");
        map.put('7', "pqrs");
        map.put('8', "tuv");
        map.put('9', "wxyz");
    char arr [] = s1.toCharArray();
    for(int i=0;i<arr.length;i++){
        for(int j=i+1;j<arr.length;j++){
            mixing()
        }
    }
    }
}
```

### ques7.java

```
/*You have a long flowerbed in which some of the plots are planted, and some are not. However, flowers cannot be
planted in adjacent plots.
Given an integer array flowerbed containing 0's and 1's, where 0 means empty and 1 means not empty, and an integer
n, return true if n new flowers can be planted in the flowerbed without violating the no-adjacent-flowers rule and
false otherwise.
Example 1:
Input: flowerbed = [1,0,0,0,1], n = 1
```

```
Output: true
Example 2:
Input: flowerbed = [1,0,0,0,1], n = 2
Output: false
Constraints:
1 <= flowerbed.length <= 2 * 104
flowerbed[i] is 0 or 1.
There are no two adjacent flowers in flowerbed.
0 <= n <= flowerbed.length*/
import java.util.*;
public class ques7 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 [] = s.nextLine().split(",");
        int arr [] = new int [s1.length];
        for(int i=0;i<s1.length;i++){
            arr[i]= Integer.parseInt(s1[i]);
        }
        int i=0;
        int n = s.nextInt();
        while(i<arr.length){
            if(n==0){
                System.out.println("True");
                return ;
            }
            if(arr[i]==0){ // can place
                if(i==0 && arr[i+1]!=1){ // starting
                    arr[i] = 1; n--; i = i+ 2;  // flower place chesi next elago avsaram ledu kabbati moving 2 steps
ahead
                }
                else if(i==arr.length-1 && arr[i-1]!=1){ // end
                    arr[i] = 1; n--; i = i+2;
                }
                else{ // anntey midlle
                    if(arr[i+1]!=1 && arr[i-1]!=1){
                        arr[i] = 1; n--; i = i+2;
                    }
                }
            }
            else{
                i++;
            }
        }
        System.out.println(Arrays.toString(arr));

    }

}
```

### ques8.java

```
/*iven an integer array nums, return true if there exists a triple of indices (i, j, k)
 such that i < j < k and nums[i] < nums[j] < nums[k]. If no such indices exists, return false.
Example 1:

Input: nums = [1,2,3,4,5]
Output: true
Explanation: Any triplet where i < j < k is valid.
Example 2:

Input: nums = [5,4,3,2,1]
Output: false
Explanation: No triplet exists.
Example 3:

Input: nums = [2,1,5,0,4,6]
Output: true
Explanation: One of the valid triplet is (1, 4, 5), because nums[1] == 1 < nums[4] == 4 < nums[5] == 6.


Constraints:

1 <= nums.length <= 5 * 105
-231 <= nums[i] <= 231 - 1*/import java.util.*;
/*idhi moddalo logic annta public class ques8 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1  = s.nextLine();
        int arr [] = new int[s1.length()];
        int l=0;
        for(char c: s1.toCharArray()){ // idhi baguundi , koncham kothaga chesina
```

```
            arr[l] = c - '0';
            l++;
        }
        //System.out.println(Arrays.toString(arr));
        int i=0,j=1,k=2;
        while(k<arr.length) {
            if(arr[i]<arr[j] && arr[i]<arr[k] && arr[j]<arr[k] ){
                System.out.print("valid");
                return;
            }
            if(arr[i] > arr[j]){
                i++; j++; k++;
            }
            if(arr[j]>arr[k]){
                k++;
            }
        }
        System.out.println("not valid");

    }

}
*/
/* ai code int first = Integer.MAX_VALUE;
int second = Integer.MAX_VALUE;

for(int num: arr){
    if(num <= first) first = num;
    else if(num <= second) second = num;
    else {
        System.out.println("valid");
        return;
    }
}

System.out.println("not valid");*/
```

### ques9.java

```
/*Given two strings s and t, return true if s is a subsequence of t, or false otherwise.
A subsequence of a string is a new string that is formed from the original
 string by deleting some (can be none) of the characters without disturbing the
 relative positions of the remaining characters.
 (i.e., "ace" is a subsequence of "abcde" while "aec" is not).
Example 1:
Input: s = "abc", t = "ahbgdc"
Output: true
Example 2:

Input: s = "axc", t = "ahbgdc"
Output: false


Constraints:

0 <= s.length <= 100
0 <= t.length <= 104
s and t consist only of lowercase English letters.


Follow up: Suppose there are lots of incoming s, say s1, s2, ..., sk where k >= 109, and you want to check one by
one to see if t has its subsequence. In this scenario, how would you change your code*/import java.util.*;
public class ques9 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        String s2 = s.nextLine();
        int i=0 , j=0;
        boolean res = app_2(s1,s2,i,j);
        System.out.print(res);
        while(i<s1.length()){
            if(j == s2.length()){
                System.out.print("true");
                return;
            }
            if(s1.charAt(i)==(s2.charAt(j))){
                i++; j++;
            }
            j++;
        }
```

```
    }
    public static boolean app_2(String s1,String s2 , int i, int j){
        if(j==s2.length()){
            return true;
        }
        if(i==s1.length()) return false;
        if(s1.length() ==  s2.length()){
            return app_2(s1,s2,i++,j++); // recursive call
        }
        else{
            return app_2(s1,s2,i,j++);
        }
    }
}
```

# may_holidays / collectionframeworks

### ques1.java

```
/*Given a list of strings, find all duplicates and return them sorted alphabetically using
Java Collections Framework.
Input:  ["apple","banana","apple","cherry","banana","apple"]
Output: ["apple", "banana"]
Constraints: Use only Collections Framework APIs.
Hint
Hide follow-up
Mark solved
Explain solution ↗
Follow-up: What is the time complexity? How would you do this with Java Streams in one pipeline? */
import java.util.*;

class ques1{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String arr [] = s.nextLine().split(" ");
        approach_1(arr);
        appraoch_2(arr);
        app_3(arr);

    }
    public static void approach_1(String arr[]){
        ArrayList <String> going = new ArrayList<>();
        TreeSet<String> res = new TreeSet<>();
        for(String x:arr){
            if(going.contains(x)){
                res.add(x);
            }
            going.add(x);
        }
        System.out.print(res);

    }
    public static void appraoch_2(String arr[]){
        HashSet <String> chustuuna = new HashSet<>();
        TreeSet <String > res = new TreeSet<>();
        for(String x: arr){
            if(!(chustuuna.add(x))){
                res.add(x);
            }
            chustuuna.add(x);

        }
        System.out.println(res);
    }
    public static void app_3(String arr[]){
        HashMap <String, Integer> m  = new HashMap<>();
        TreeSet <String> res = new TreeSet<>();
        for (String x : arr){
            m.put(x,m.getOrDefault(x,0) +1);
        }
        //System.out.println(m);
        for(String x : m.keySet()){
            if(m.get(x)>1) res.add(x);
        }
        System.out.println(res);
    }
}
```

### ques2.java

```

```

# may_holidays / recurssion

### ques1.java

```
/*mplement binary search recursively. Return the index if the element is found, else return -1.
Input:  arr = [1, 3, 5, 7, 9, 11], target = 7
Output: 3
Constraints: Array is sorted. O(log n) time.
Hint
Hide follow-up
Mark solved
Explain solution ↗
Follow-up: How does recursion depth compare to iterative approach? Which is preferred in production? */
import java.util.*;
class ques1{
    public static void main(String args[])   {
        Scanner s = new Scanner(System.in);
        String s1 [] = s.nextLine().split(" ");
        int arr [] = new int [s1.length];
        for(int i=0;i<s1.length;i++){
            arr[i] = Integer.parseInt(s1[i]);
        }
        int k = s.nextInt();
    int low = 0 , high = s1.length - 1;
    int res = binary(arr,k,low, high);
    System.out.print(res);
    }
    public static int binary(int arr[], int k,int low, int high){
        int mid = (low + high)/2;
        if(low > high) return -1;
        if(arr[mid] == k) return mid;

        else if (arr[mid] < k){
            return binary(arr,k,mid +1,high);
        }
        else {
           return  binary(arr,k,low,mid-1);
        }
    }
}
```

### ques2.java

```
/*Implement the myAtoi(string s) function, which converts a string to a 32-bit signed integer.

The algorithm for myAtoi(string s) is as follows:

Whitespace: Ignore any leading whitespace (" ").
Signedness: Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither
present.
Conversion: Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the
string is reached. If no digits were read, then the result is 0.
Rounding: If the integer is out of the 32-bit signed integer range [-231, 231 - 1], then round the integer to remain
in the range. Specifically, integers less than -231 should be rounded to -231, and integers greater than 231 - 1
should be rounded to 231 - 1.
Return the integer as the final result.



Example 1:

Input: s = "42"

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.
Step 1: "42" (no characters read because there is no leading whitespace)
         ^
Step 2: "42" (no characters read because there is neither a '-' nor '+')
          ^
Step 3: "42" ("42" is read in)
            ^
Example 2:

Input: s = " -042"

Output: -42
```

```
Explanation:

Step 1: "   -042" (leading whitespace is read and ignored)
           ^
Step 2: "   -042" ('-' is read, so the result should be negative)
            ^
Step 3: "   -042" ("042" is read in, leading zeros ignored in the result)
              ^
Example 3:

Input: s = "1337c0d3"

Output: 1337

Explanation:

Step 1: "1337c0d3" (no characters read because there is no leading whitespace)
         ^
Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')
         ^
Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)
              ^
Example 4:

Input: s = "0-1"

Output: 0

Explanation:

Step 1: "0-1" (no characters read because there is no leading whitespace)
         ^
Step 2: "0-1" (no characters read because there is neither a '-' nor '+')
         ^
Step 3: "0-1" ("0" is read in; reading stops because the next character is a non-digit)
          ^
Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.



Constraints:

0 <= s.length <= 200
s consists of English letters (lower-case and upper-case), digits (0-9), ' ', '+', '-', and '.'. */
```

### substring.java

```
//package java.interviews.recurssion;
public class substring {
    public static void main(String args[]){
        rev("ab");
    }
    public static void rev(String k){

    }
}
```

# may_holidays / sliding window

### ques1.java

```
/*Given an array of positive integers and a number k, find the maximum sum of any contiguous subarray of size k.
Input:  arr = [2, 1, 5, 1, 3, 2], k = 3
Output: 9   // subarray [5, 1, 3]
Constraints: O(n) time. Do not use nested loops.
Hint
Hide follow-up
Mark solved
Explain solution ↗
Follow-up: What if k is larger than the array length? What if array has negative numbers? */
import java.util.*;
public class ques1{
    public static void main(String args[]){
        int a [] =  {2, 1, 5, 1, 3, 2};
        int k = 3, sum = 0;
        for(int i=0;i<k;i++){
            sum = sum + a[i];
        }
        int maxsum=Integer.MIN_VALUE;
        maxsum = Math.max(maxsum,sum);
        // ila show chese badalu direct ga maxsum = sum pettachu
        int ans = 0;
        for(int i=k;i<a.length;i++){
            sum = sum - a[i-k] + a[i];
            maxsum = Math.max(maxsum,sum);

        }
        System.out.print(maxsum);
    }
}
```

### ques2.java

```
/*Find the longest subarray with sum equal to k.
Input:  arr = [10, 5, 2, 7, 1, 9], k = 15
Output: 4  // subarray [5, 2, 7, 1]
Constraints: Array can have positive integers. O(n) time. */
import java.util.*;
public class ques2{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        s1 = s1.replace("[","");
        s1 = s1.replace("]","");
        System.out.print(s1);
        String s2 [] = s1.split(",");
        int arr [] = new int[s2.length];
        for(int i=0;i<s2.length;i++){
            arr[i] = Integer.parseInt(s2[i]);
        }
        int k = s.nextInt();
        int sum = 0; int l= 1; int j = 0; int i=0;
        int maxl = Integer.MIN_VALUE;
        while(i<arr.length){
            if(sum == k){
                maxl = Math.max(maxl , l);
                sum = sum + arr[i]; l++; i++;
            }
            else if(sum < k){
                sum = sum + arr[i]; l++; i++;
            }
            else {
                sum = sum - arr[j];
                j++; l--;
            }

        }
        System.out.println(maxl);
    }
}
```

### ques3.java

```
/*Given an array of daily temperatures and an integer k=3, return the maximum
average temperature over any k consecutive days*/
import java.util.*;
public class ques3 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = s.nextInt();
        }
        int k = s.nextInt();
        int sum = 0;
        for(int i=0;i<k;i++){
            sum = sum + arr[i];
        }
        int m = sum;
        for(int i=k;i<arr.length;i++){
            sum += arr[i];
            sum -= arr[i-k];
            m = Math.max(sum,m);
        }
        System.out.println(m/k);
    }

}
```

### tempCodeRunnerFile.java

```
[10, 5, 2, 7, 1, 9]
```

# may_holidays / streams

### ques1.java

```
/*Given a list of integers, use Java Streams to find the count, sum, average, min, and max in a single pipeline.
Input:  [3, 1, 4, 1, 5, 9, 2, 6]
Output: count=8, sum=31, avg=3.875, min=1, max=9
Constraints: Single stream pass using IntSummaryStatistics.
Hint
Hide follow-up
Mark solved
Explain solution ↗
Follow-up: Why can you not reuse a Stream after a terminal operation? */
import java.util.*;
public class ques1{
    public static void main(String args[]){
        ArrayList <Integer> l = new ArrayList<>((Arrays.asList(3, 1, 4, 1, 5, 9, 2, 6)));
        System.out.print(l.stream().count());
       l.stream().foreach(n -> sum + n);
    }
}
```

# may_holidays / strings

### ques1.java

```
/* Write a function to reverse the words in a sentence without
 reversing characters inside each word.
Example:
Input:  "the sky is blue"
Output: "blue is sky the"
Constraints:
Do not use split()
Handle multiple spaces, leading/trailing spaces
Optimize for time and space */
//approach 1:
import java.util.*;
public class ques1{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        approach_1(s1);
        approach_2(s1);
        appraoch_3(s1);
        appraoch_4(s1);
    }
        public static void approach_1(String s1){
        List <String> l = new ArrayList<>();
        s1 = s1.trim();
        String s2 = "";
        for(int i=0;i<s1.length();i++){
            if(s1.charAt(i)!=' '){
                s2 = s2+s1.charAt(i);
            }
            else{
                if(!s2.equals("")){ // very imp line
                s2 = s2.trim();
                l.add(s2);
                s2 = "";
                }
            }
        }
        if(!s2.equals("")){ //for last word
            s2 = s2.trim();
            l.add(s2);
        }

        String arr [] = l.toArray(new String[0]);
        String res = "";
        for(int i=arr.length-1;i>=0;i--){
            res = res + arr[i];
            res = res + " ";
        }
        System.out.println(res);

    }
    public static void approach_2(String s1){

       String res = rev(s1);
        //System.out.print(res);
    // now reverse each word
    String s2 = "";
    String res3 = "";
    for(int k=0;k<res.length();k++){
        if(res.charAt(k) != ' '){
            s2 = s2 + res.charAt(k);
        }
        else{
            if(!s2.equals("")){
                String res2 = rev(s2);
                s2 = "";
                res3 = res3 + res2;
                res3 = res3 + " ";
            }
        }


    }
    if(!s2.equals("")){
        String res2 = rev(s2);
        s2 = "";
        res3 = res3 + res2;
    }
        System.out.print(res3);
```

```
    }
    public static String rev(String s2){
        int i=0; int j = s2.length() -1 ;
        char [] arr2 = s2.toCharArray();
        while(i<j){
            char temp = arr2[i];
            arr2[i] = arr2[j];
            arr2[j] = temp;
            i++ ; j-- ;
        }
        String res = new String(arr2);
        return res;
    }
public static void appraoch_3(String s1){
    StringBuilder sha = new StringBuilder();
    int i=0 , n = s1.length();
    while(i<n  && s1.charAt(i)== ' '){
        i++ ;                                   // starting spaces
    }
    while(i<n){
        while(i<n && s1.charAt(i)!= ' '){
            sha.append(s1.charAt(i)); // only taking wanted
            i++;
        }
        while(i<n && s1.charAt(i) == ' '){
            i++;
        }
        if(i<n){ // anntey there are more words
            sha.append(' '); // we are adding one space after every word
        }

    }
    System.out.print(sha);
    // now lets play :
    String clean = sha.toString();
    String res = rev(clean);
    char [] arr = res.toCharArray();
    int left = 0, l = 0, right = 0;
    while(l<arr.length){
        while(l<arr.length && arr[l]!= ' '){
            l++;
        }
        right = l - 1;
        rev_in_place(arr,left,right);
        left = l + 1 ;
        l++;
    }
    System.out.print(new String(arr));

}
public static void rev_in_place(char arr[], int left, int right){
    while(left < right){
        char temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
        left++; right--;
    }
}

public static void appraoch_4(String s1){
    Stack <String> st = new Stack<>();
    // lets clean it
    StringBuilder sb = new StringBuilder();
    int i=0; int n = s1.length();
    while (i<n && s1.charAt(i)==' '){
        i++; // starting spaces

    }
    while(i<n){
        while(i<n && s1.charAt(i)!=' '){
            sb.append(s1.charAt(i));
            i++;
        }
        while(i<n && s1.charAt(i) == ' '){
            i++;
        }
        if(i<n){
            sb.append(' '); // space after each word
            //System.out.println("buffer now : "+sb);

        }
    }
```

```
    //System.out.println("buffer now:"+sb);
    StringBuilder sb2 = new StringBuilder();
    for(int k=0;k<sb.length();k++ ){
        while ( k<sb.length()&& sb.charAt(k) != ' '){
            sb2.append(sb.charAt(k));
            k++;
        }
        st.push(sb2.toString());
        sb2.setLength(0);

    }
StringBuilder res = new StringBuilder();
System.out.println("stack : "+st);
while(!(st.empty())){
    res.append(st.pop());
    res.append(' ');
}
System.out.println(res);

}
}
```

### ques2.java

```
/*Check whether a given string is a valid palindrome considering only alphanumeric
 characters and ignoring case.
Input:  "A man, a plan, a canal: Panama"
Output: true

Input:  "race a car"
Output: false
Constraints: O(n) time, O(1) space. Do not use extra string or StringBuilder. */
import java.util.*;
class ques2{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        int i=0, j =s1.length()-1;
        while(i<j){
            char ci = Character.toLowerCase(s1.charAt(i));
            char cj = Character.toLowerCase(s1.charAt(j));
            if(!(Character.isLetterOrDigit(ci))){
                i++;
                continue; // ivi chala imp
            }
            if(!(Character.isLetterOrDigit(cj))){
                j--;
                continue;
            }
            if(ci != cj){
                System.out.println(ci); System.out.println(i);
                System.out.print(cj); System.out.println(j);
                System.out.println("not palindriome");
                return;
            }
            i++; j--;
        }
        System.out.println("palimdrome");
    }
}
```

### ques3.java

```
/*Given a string s, reverse only all the vowels in the string and return it.

The vowels are 'a', 'e', 'i', 'o', and 'u', and they can appear in both lower and upper cases, more than once.



Example 1:

Input: s = "IceCreAm"

Output: "AceCreIm"

Explanation:

The vowels in s are ['I', 'e', 'e', 'A']. On reversing the vowels, s becomes "AceCreIm".

Example 2:
```

```
Input: s = "leetcode"

Output: "leotcede"



Constraints:

1 <= s.length <= 3 * 105
s consist of printable ASCII characters.*/import java.util.*;
public class ques3 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
       String s1 = s.nextLine();
       char arr [] = s1.toCharArray();
       int i=0,j=arr.length - 1;
       while(i<j){
        boolean k = (arr[i]=='a' || arr[i] == 'e' || arr[i] == 'i' || arr[i] == 'o' || arr[i] == 'u' || arr[i]== 'A'
|| arr[i] == 'E' ||arr[i] == 'O' ||arr[i] == 'I' || arr[i] == 'U');
        boolean l = (arr[j]=='a' || arr[j]=='e' || arr[j]=='i' || arr[j]=='o' || arr[j]=='u'
          || arr[j]=='A' || arr[j]=='E' || arr[j]=='I' || arr[j]=='O' || arr[j]=='U');
            if(!k)
                i++;
            else if(!l) j--;
            else if(!k && !l){ i++; j--;}
            else{
                char temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp ;
                i++; j--;
            }
       }
       System.out.println(new String (arr));
    }

}
```

### ques4.java

```
/*You are given two strings word1 and word2. Merge the strings by adding letters in alternating order, starting with
word1. If a string is longer than
 the other, append the additional letters onto the end of the merged string.
Return the merged string.
Example 1:

Input: word1 = "abc", word2 = "pqr"
Output: "apbqcr"
Explanation: The merged string will be merged as so:
word1:  a   b   c
word2:    p   q   r
merged: a p b q c r
Example 2:

Input: word1 = "ab", word2 = "pqrs"
Output: "apbqrs"
Explanation: Notice that as word2 is longer, "rs" is appended to the end.
word1:  a   b
word2:    p   q   r   s
merged: a p b q   r   s
Example 3:

Input: word1 = "abcd", word2 = "pq"
Output: "apbqcd"
Explanation: Notice that as word1 is longer, "cd" is appended to the end.
word1:  a   b   c   d
word2:    p   q
merged: a p b q c   d


Constraints:

1 <= word1.length, word2.length <= 100
word1 and word2 consist of lowercase English letters.*/import java.util.*;
public class ques4 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        String s2 = s.nextLine();
        StringBuilder res = new StringBuilder();
        int n = Math.min(s1.length(),s2.length());
```

```
        for(int i=0;i<n;i++){
            res.append(s1.charAt(i)); res.append(s2.charAt(i));

        }
        if(s1.length()>n){

            res.append(s1.substring(n));
        }
        else if(s2.length()>n){
            res.append(s2.substring(n));
        }
        System.out.print(res);
    }

}
```

### ques5.java

```
/*Given an array of characters chars, compress it using the following algorithm:

Begin with an empty string s. For each group of consecutive repeating characters in chars:

If the group's length is 1, append the character to s.
Otherwise, append the character followed by the group's length.
The compressed string s should not be returned separately,
 but instead, be stored in the input character array chars.
  Note that group lengths that are 10 or longer will be split into multiple characters in chars.

After you are done modifying the input array, return the new length of the array.

You must write an algorithm that uses only constant extra space.

Note: The characters in the array beyond the returned length do not matter and should be ignored.



Example 1:

Input: chars = ["a","a","b","b","c","c","c"]
Output: 6
Explanation: The groups are "aa", "bb", and "ccc". This compresses to "a2b2c3".
Example 2:

Input: chars = ["a"]
Output: 1
Explanation: The only group is "a", which remains uncompressed since it's a single character.
Example 3:

Input: chars = ["a","b","b","b","b","b","b","b","b","b","b","b","b"]
Output: 4
Explanation: The groups are "a" and "bbbbbbbbbbbb". This compresses to "ab12".*/
/* idhi code cheyatam koncham agam agam chesinga
import java.util.*;
public class ques5 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        char arr [] = s1.toCharArray();
        int i=0,j=1,w=0;
        while(i<arr.length){
            if(j<arr.length && arr[i]==arr[j]) j++;
            else{
                if((j-i)>1){
                    arr[w]=arr[i];
                    String count = String.valueOf((j-i));
                    for(char c: count.toCharArray()){
                            w++;
                            arr[w]=c;
                    }
                    i = j;
                    j++;
                }
                else{
                    w++;
                    arr[w] = arr[i];
                    i++; j++;
                }
            }
        }
        System.out.print(new String(arr));
    }
```

```
}
*/
/*ai
int i = 0, w = 0;
int n = arr.length;

while(i < n) {
    char current = arr[i];
    int j = i;

    // scan the group
    while(j < n && arr[j] == current) {
        j++;
    }

    int count = j - i;

    // write character
    arr[w++] = current;

    // write count if more than 1
    if(count > 1){
        for(char c: String.valueOf(count).toCharArray()){
            arr[w++] = c;
        }
    }

    // move i to next group
    i = j;
}*/
```

### ques6.java

```
/*For two strings s and t, we say "t divides s" if and only if s = t + t + t + ... + t + t (i.e., t is concatenated
with itself one or more times).

Given two strings str1 and str2, return the largest string x such that x divides both str1 and str2.



Example 1:

Input: str1 = "ABCABC", str2 = "ABC"

Output: "ABC"

Example 2:

Input: str1 = "ABABAB", str2 = "ABAB"

Output: "AB"

Example 3:

Input: str1 = "LEET", str2 = "CODE"

Output: ""

Example 4:

Input: str1 = "AAAAAB", str2 = "AAA"

Output: ""



Constraints:

1 <= str1.length, str2.length <= 1000*/
import java.util.*;
public class ques6 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        String s2 = s.nextLine();
        System.out.print(app_1(s1,s2));
        System.out.print(app_2(s1,s2));
    }
    public static String app_1(String s1,String s2){
        int n = s1.length();
        int m= s2.length();
```

```
        int l = Math.min(s1.length(),s2.length());
        String t = "";
        if(n>m){
             t = s2;
        }
        else {
            t = s1;
        }
        while(l>0){
            if(n%l == 0 && m%l == 0){ // anntey possiblity unndi
                t = t.substring(0,l);
                String k = t.repeat(n/l);
                String j = t.repeat(m/l);
                if(k.equals(s1) && j.equals(s2)){
                    return t;
                }
            }
            l--;
        }
        return "";
    }
    public static String app_2(String s1, String s2){
        if(!(s1+s2).equals(s2+s1)) return "";
        int gcd = gcd(s1.length(),s2.length());
        return s1.substring(0,gcd);
    }
    public static int gcd(int a, int b){
        if(b==0) return a;
        else{
            return gcd(b,a%b);
        }
    }

}
```

### ques7.java

```
/*Given two strings s and t, return true if s is a subsequence of t, or false otherwise.

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of
the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence
of "abcde" while "aec" is not).



Example 1:

Input: s = "abc", t = "ahbgdc"
Output: true
Example 2:

Input: s = "axc", t = "ahbgdc"
Output: false */
import java.util.*;
public class ques7{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        String t = s.nextLine();
        if(s1.length() >t.length()){{
            return ;
        }
        }
        int i=0,j=0;
        boolean res = app_2(s1,t,i,j);
        System.out.print(res);
        while(t.length()>j){
            if(s1.length()==i){
                System.out.print("true");
                return;
            }
            if(s1.charAt(i)==t.charAt(j)){
                i++;j++;
            }
            else{
                j++;
            }

        }
        if(s1.length()==i){
            System.out.print("True"); // this is an imp case
```

```
        }
        else{
            System.out.print("False");
        }

    }
    public static boolean app_2(String s1,String t,int i,int j){
        if(s1.length()==i){
            return true;
        }
        else if (t.length()==j){
            return false;
        }
        else if(s1.charAt(i)==t.charAt(j)){
            //return app_2(s1,t,i++,j++); // mistake
            return app_2(s1,t,i+1,j+1);
        }
        else{
            return app_2(s1,t,i,j+1);
        }
    }
}
```

### ques8.java

```
/*A valid parentheses string is either empty "", "(" + A + ")", or A + B, where A and B are valid parentheses
strings, and + represents string concatenation.

For example, "", "()", "(())()", and "(()(()))" are all valid parentheses strings.
A valid parentheses string s is primitive if it is nonempty, and there does not exist a way to split it into s = A +
B, with A and B nonempty valid parentheses strings.

Given a valid parentheses string s, consider its primitive decomposition: s = P1 + P2 + ... + Pk, where Pi are
primitive valid parentheses strings.

Return s after removing the outermost parentheses of every primitive string in the primitive decomposition of s.



Example 1:

Input: s = "(()())(())"
Output: "()()()"
Explanation:
The input string is "(()())(())", with primitive decomposition "(()())" + "(())".
After removing outer parentheses of each part, this is "()()" + "()" = "()()()".
Example 2:

Input: s = "(()())(())(()(()))"
Output: "()()()()(())"
Explanation:
The input string is "(()())(())(()(()))", with primitive decomposition "(()())" + "(())" + "(()(()))".
After removing outer parentheses of each part, this is "()()" + "()" + "()(())" = "()()()()(())".
Example 3:

Input: s = "()()"
Output: ""
Explanation:
The input string is "()()", with primitive decomposition "()" + "()".
After removing outer parentheses of each part, this is "" + "" = "".


Constraints:

1 <= s.length <= 105
s[i] is either '(' or ')'.
s is a valid parentheses string.*/ import java.util.*;
public class ques8 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String s1 = s.nextLine();
        System.out.println(app_1(s1));
        System.out.print(app_2(s1));

    }
    public static String app_1(String s1){
        StringBuilder res = new StringBuilder();
        Stack<Character> st = new Stack<>();
        for(char c: s1.toCharArray()){
            if(c == '('){
                if(st.size() > 0){
```

```
                    res.append(c);
                }
                st.push(c);
            }
            else{
                if(st.size()> 1){
                    res.append(c);
                }
                st.pop();
            }
        }
        return res.toString();
    }
    public static StringBuilder app_2(String s1){ // dont do this , nen edo oorikey chesina , > **"Build with
StringBuilder, return as String."**
        StringBuilder res = new StringBuilder();
        int depth = 0;
        for(char c : s1.toCharArray()){
            if(c == '('){
                if(depth > 0) res.append(c);

            depth++;
            }
            else{
                if(depth>1) res.append(c);
                depth --;
            }
        }
        return res;

    }

}
```

# may_holidays / two ponters

### ques1.java

```
/*Given an integer array, sort it containing 0s, 1s, and 2s in-place without using any sort function.
Input:  [2, 0, 2, 1, 1, 0]
Output: [0, 0, 1, 1, 2, 2]
Constraints: One-pass, O(1) extra space. (Dutch National Flag problem)
Hint
Hide follow-up
Mark solved
Explain solution ↗
Follow-up: Can you do this in a single pass? What is the loop invariant? */
import java.util.*;
public class ques1{
    public static void main(String args[]){
        int arr []= {2,0,2,1,1,0};
        int i=0, j = arr.length -1;
        while(i<j){
            if(arr[i]>arr[j]){
                if(arr[i] == 2 && arr[j] == 0){
                    int temp =
                }
            }
        }
    }
}
```

### rev.java

```

import java.util.Arrays;

/*Problem: Reverse an Array / String (Two Pointer Technique)
Problem Statement:
You are given an array of integers or a string.
Your task is to reverse it in-place using the two pointer technique.
You are not allowed to use any extra array or built-in reverse functions.
Input:
1. An integer array arr of size n
   OR
2. A string s
Output:
Return the reversed array or string.
Examples:
Example 1:
Input:
arr = [1, 2, 3, 4, 5]
Output:
[5, 4, 3, 2, 1]
Example 2:
Input:
s = "shivaya"
Output:
"ayavihs"
Constraints:
1 ≤ n ≤ 10^5
-10^9 ≤ arr[i] ≤ 10^9
Expected Time Complexity:
O(n)
Expected Space Complexity:
O(1)
Approach Hint:
Use two pointers:
- One pointer starts from the beginning
- One pointer starts from the end
Swap the elements and move the pointers towards each other.
*/
public class rev {
    public static void main(String[] args) {
        int [] a= {4,5,6,7,8};
        int i = 0,j=a.length-1;
        while(i<j){
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            i++;
            j--;
        }
       System.out.println(Arrays.toString(a));
```

```
    }
}


public class ReverseArray {
    public static void main(String[] args) {
        char[] bestTeacher = {'S','h','a','s','h','a','n','k'};

        // Print Original
        System.out.print("Original: ");
        for (int x = 0; x < bestTeacher.length; x++) {
            System.out.print(bestTeacher[x]);
        }
        System.out.println();
        int i = 0;
        int j = bestTeacher.length - 1;
        // Reverse using two pointers
        while (i < j) {
            char temp = bestTeacher[i];
            bestTeacher[i] = bestTeacher[j];
            bestTeacher[j] = temp;

            i++;
            j--;
        }
        // Print Reversed
        System.out.print("Reversed: ");
        for (int x = 0; x < bestTeacher.length; x++) {
            System.out.print(bestTeacher[x]);
        }
    }
}
```

### water.java

```
/*Problem: Container With Most Water (Two Pointer Technique)
Problem Statement:
You are given an integer array height of length n.
Each element represents the height of a vertical line drawn at that index.
Find two lines that, together with the x-axis, form a container such that the
container holds the maximum amount of water.
Note:
You may not tilt the container.
Input:
An integer array height of size n.
Output:
Return an integer representing the maximum amount of water that can be stored.
Examples:
Example 1:
Input:
height = [1,8,6,2,5,4,8,3,7]
Output:
49

Explanation:
The maximum area is obtained by choosing lines at index 1 and 8.
Width = 8 - 1 = 7
Height = min(8, 7) = 7
Area = 7 * 7 = 49

Example 2:
Input:
height = [1,1]

Output:
1

Constraints:
2 ≤ n ≤ 10^5
0 ≤ height[i] ≤ 10^4

Expected Time Complexity:
O(n)

Expected Space Complexity:
O(1)

Approach Hint:
- Use two pointers: one at the start and one at the end
```

```
- Calculate ar
 */
public class water {

}
```

# questions_from_world

### 1.java

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        int l = 0;
        int sum = 0;
        int count = 0;
        for(int r=0;r<nums.length;r++){
            sum += nums[r];
            while(sum >k) {
                sum -= nums[l];
                l++;
            }
            if(sum == k) count ++;
        }
        return count;
    }
}
```

### Answer_Queries_with_Sum_of_Elements_2389.java

```
/*/*
Question:

Given an integer array nums and query values,
find the maximum length of a subsequence whose sum
is less than or equal to each query.

Input Format:
First line contains n.
Second line contains n array elements.
Third line contains q.
Fourth line contains q query values.

Output Format:
Print the maximum subsequence length for each query.

Sample Input:
4
4 5 2 1
3
3 10 21

Sample Output:
2 3 4
*/
import java.util.*;
public class Answer_Queries_with_Sum_of_Elements_2389{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int nums [] = new int [n];
        for(int i=0;i<n;i++) nums[i] = s.nextInt();
        int q = s.nextInt();
        int query [] = new int [q];
        for(int i=0;i<q;i++) query[i] = s.nextInt();
        int res [] = new int [q];
        int i=0;
        for(int x:query){
            res[i] = calc_max_sequence(nums,x);
            i++;
        }
        System.out.print(Arrays.toString(res));

    }
    public static int calc_max_sequence(int nums[],int target){
        int left = 0, right = nums.length-1;
        int sum =0;
        int res =0 ;
        for(right = 0;right<nums.length;right++){
            sum += nums[right];
            while(sum>target && left <nums.length){
                sum-= nums[left];
                left++;
            }
            res = Math.max(res,right-left+1);

                }
```

```
                return res;

    }
}
```

### Sort_Array_By_Parity_905.java

```
public class Sort_Array_By_Parity_905{
    public int[] sortArrayByParity(int[] nums) {
        int res [] = new int[nums.length];
        int i = 0, j= nums.length-1;
        for (int x :nums){
            if(x %2 == 0) {
                res[i] = x;
                i++;
            }
            else{
                res[j] = x; j--;
            }
        }
        return res;
    }
}
```

# striver / arrays / faq_medium

### leaders_in_a_array.java

```
/*Given an integer array nums, return a list of all the leaders in the array.

A leader in an array is an element whose value is strictly greater than all elements to its right in the given
array.
The rightmost element is always a leader. The elements in the leader array must appear in the order they appear in
the nums array.

Example 1:
Input: nums = [1, 2, 5, 3, 1, 2]

Output: [5, 3, 2]

Explanation:

2 is the rightmost element, 3 is the largest element in the index range [3, 5], 5 is the largest element in the
index range [2, 5]

Example 2:
Input: nums = [-3, 4, 5, 1, -4, -5]

Output: [5, 1, -4, -5]

Explanation:

-5 is the rightmost element, -4 is the largest element in the index range [4, 5], 1 is the largest element in the
index range [3, 5] and 5 is the largest element in the range [2, 5]

Now Your Turn!
Pick the correct output for the given input
Input: nums = [-3, 4, 5, 1, -30, -10]


[-3, 4, 5]

[5, 1, -30]

[5, 1, -10]

[5, -10, 1]
Still unsure what the problem is asking ?

Let’s go through a few more examples, step by step, to make it clearer.

Constraints:
1 <= nums.length <= 105
-104 <= nums[i] <= 104*/
import java.util.*;
public class leaders_in_a_array {
    public static void main(String args[]){
        Scanner s= new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int [n];
        for (int i=0;i<n;i++) arr[i] = s.nextInt();
        System.out.println(app1(arr));
        System.out.println(brinda(arr));

    }
    public static ArrayList<Integer> app1(int arr[]){
        ArrayList<Integer> res = new ArrayList<>();
        for(int i=0;i<arr.length;i++){ // last varaku check cheayali
            if(can_leader(arr,i)){
                //System.out.println(res + " " +i);
                res.add(arr[i]);
            }
        }
        return res;
    }
    public static boolean can_leader(int arr[],int i){
        for (int j = i+1;j<arr.length;j++){
            if(arr[j]>=arr[i]) return false;
        }
        return true;
    }
    public static ArrayList<Integer> brinda(int arr[]){
        ArrayList<Integer> res = new ArrayList<>();
        int max = arr[arr.length-1];
        res.add(max);
        for(int j=arr.length-2;j>=0;j--){
```

```
            if(arr[j]>max){
                max = arr[j];
                res.add(max);
            }
        }
        Collections.reverse(res);
        return res;
}
}
//https://claude.ai/share/cf1e05b5-824c-4c74-b27d-8e1c4797ec62
```

### Pascals_Triangle_3.java

```
/*Given an integer n, return the first n (1-Indexed) rows of Pascal's triangle.

In Pascal's triangle:

The first row has one element with a value of 1.
Each row has one more element in it than its previous row.
The value of each element is equal to the sum of the elements directly above it when arranged in a triangle format.
Example 1:
Input: n = 4

Output: [[1], [1, 1], [1, 2, 1], [1, 3, 3, 1]]

Explanation: The Pascal's Triangle is as follows:

1

1 1

1 2 1

1 3 3 1

1st Row has its value set to 1.

All other cells take their value as the sum of the values directly above them

Example 2:
Input: n = 5

Output: [[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]

Explanation: The Pascal's Triangle is as follows:

1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

1st Row has its value set to 1.

All other cells take their value as the sum of the values directly above them*/
import java.util.*;
public class Pascals_Triangle_3 {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int r = s.nextInt();
        ArrayList<ArrayList<Integer>> pascal = build_pascal(r); // ofc no of rows ee kavali kada
        System.out.print(pascal);
        //System.out.print(pascal.get(r-1).get(c-1));
    }
    public static ArrayList<ArrayList<Integer>> build_pascal(int n){
         ArrayList<ArrayList<Integer>> res = new ArrayList<>();
         for(int i=0;i<n;i++){
            ArrayList<Integer> current = new ArrayList<>();
            current.add(1); // start index

            for(int j=1;j<=i-1;j++){
                 ArrayList<Integer> prev = res.get(i-1); // pathadi pattuchi vadali kada
                 current.add(prev.get(j)+prev.get(j-1));
            }
            if(i>0) current.add(1);
            res.add(current);
         }
```

```
        return res;
    }
}
//https://claude.ai/share/c275341e-7ec4-46f8-80a3-830fa71c620f
```

### Pascals_Triangle_I.java

```
/*Given two integers r and c, return the value at the rth row and cth column (1-indexed) in a Pascal's Triangle.

In Pascal's triangle:

The first row contains a single element 1.
Each row has one more element than the previous row.
Every row starts and ends with 1.
For all interior elements (i.e., not at the ends), the value at position (r, c) is computed as the sum of the two
elements directly above it from the previous row:
Pascal[r][c]=Pascal[r−1][c−1]+Pascal[r−1][c]
where indexing is 1-based
Example 1:
Input: r = 4, c = 2

Output: 3

Explanation:

The Pascal's Triangle is as follows:

1

1 1

1 2 1

1 3 3 1

....

Thus, value at row 4 and column 2 = 3

Example 2:
Input: r = 5, c = 3

Output: 6

Explanation:

The Pascal's Triangle is as follows:

1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

....

Thus, value at row 5 and column 3 = 6*/
import java.util.*;
public class Pascals_Triangle_I {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int r = s.nextInt();
        int c = s.nextInt();
        ArrayList<ArrayList<Integer>> pascal = build_pascal(r); // ofc no of rows ee kavali kada
        //System.out.print(pascal);
        System.out.print(pascal.get(r-1).get(c-1));
    }
    public static ArrayList<ArrayList<Integer>> build_pascal(int n){
         ArrayList<ArrayList<Integer>> res = new ArrayList<>();
         for(int i=0;i<n;i++){
            ArrayList<Integer> current = new ArrayList<>();
            current.add(1); // start index

            for(int j=1;j<=i-1;j++){
                 ArrayList<Integer> prev = res.get(i-1); // pathadi pattuchi vadali kada
                 current.add(prev.get(j)+prev.get(j-1));
            }
```

```
            if(i>0) current.add(1);
            res.add(current);
         }
         return res;
    }
}
```

### Rearrange_by_Sign.java

```
/*Given an integer array nums of even length consisting of an equal number of positive and negative integers.Return
the answer array in such a way that the given conditions are met:

Every consecutive pair of integers have opposite signs.
For all integers with the same sign, the order in which they were present in nums is preserved.
The rearranged array begins with a positive integer.
Example 1:
Input : nums = [2, 4, 5, -1, -3, -4]

Output : [2, -1, 4, -3, 5, -4]

Explanation:

The positive number 2, 4, 5 maintain their relative positions and -1, -3, -4 maintain their relative positions

Example 2:
Input : nums = [1, -1, -3, -4, 2, 3]

Output : [1, -1, 2, -3, 3, -4]

Explanation:

The positive number 1, 2, 3 maintain their relative positions and -1, -3, -4 maintain their relative positions */

import java.util.Arrays;
import java.util.Scanner;
public class Rearrange_by_Sign{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr[] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        System.out.print(Arrays.toString(app1(arr))); //brute
        System.out.print(Arrays.toString(app2(arr))); //better
        System.out.print(Arrays.toString(app3(arr))); //optimal

    }

    public static int[] app1(int arr[]) {

        int n = arr.length;

        int pos[] = new int[n / 2];
        int neg[] = new int[n / 2];

        int a = 0, b = 0;

        // Separate positive and negative numbers
        for(int x : arr) {

            if(x > 0) {
                pos[a++] = x;
            }
            else {
                neg[b++] = x;
            }
        }

        int res[] = new int[n];

        int p = 0, q = 0, k = 0;

        // Alternate positive and negative
        while(p < pos.length && q < neg.length) {

            res[k++] = pos[p++];
            res[k++] = neg[q++];
        }

        return res;
    }
```

```
public static int[] app2(int[] arr) {
    int[] res = new int[arr.length];

    int i = 0;
    for (int x : arr) {              // pass 1: positives at 0, 2, 4...
        if (x > 0) { res[i] = x; i += 2; }
    }

    int j = 1;
    for (int x : arr) {              // pass 2: negatives at 1, 3, 5...
        if (x < 0) { res[j] = x; j += 2; }
    }
    return res;
}
    public static int [] app3(int arr[]){
        int i = 0;int j=1;
        int res [] = new int[arr.length];
        for(int x:arr){
            if(x>0) {
                res[i] = x;
                i = i+2;
        }
        else{
            res[j] = x;
            j = j+2;
        }
    }
    return res;
}
}
```

### Rotate_Matrix_by_90_Degrees.java

```
/*Given an N * N 2D integer matrix, rotate the matrix by 90 degrees clockwise.

The rotation must be done in place, meaning the input 2D matrix must be modified directly.

Example 1:
Input: matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
Output: matrix = [[7, 4, 1], [8, 5, 2], [9, 6, 3]]
Example 2:
Input: matrix = [[0, 1, 1, 2], [2, 0, 3, 1], [4, 5, 0, 5], [5, 6, 7, 0]]
Output: matrix = [[5, 4, 2, 0], [6, 5, 0, 1], [7, 0, 3, 1], [0, 5, 1, 2]]
Now Your Turn!
Pick the correct output for the given input
Input: matrix = [[1, 1, 2], [5, 3, 1], [5, 3, 5]]
[[2, 1, 5], [1, 3, 3], [1, 5, 5]]
[[5,3, 5], [1, 3, 5], [2, 1, 1]]
[[5, 5, 1], [3, 3, 1], [5, 1, 2]]
[[5, 3, 5], [1, 3, 5], [1, 1, 2]]
Still unsure what the problem is asking ?

Let’s go through a few more examples, step by step, to make it clearer.

Constraints:
n == matrix.length.
n == matrix[i].length.
1 <= n <= 100.
-104 <= matrix[i][j] <= 104
Advertisements

*/
// ee code array laga adigthey ney

import java.util.*;
public class Rotate_Matrix_by_90_Degrees {
public static void main(String args[])    {
    Scanner s = new Scanner(System.in);
    int n = s.nextInt();
    int arr [][] = new int[n][n];
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++) arr[i][j] = s.nextInt();
    }
    app1(arr,n);
    for (int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            System.out.print(arr[i][j]+" ");
        }
        System.out.println();
    }
}
```

```
public static int [][] app1(int arr[][], int n){
    // transpose cheddam :
    for (int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){ // i+1 anta not zero (sad)
            int temp = arr[i][j];
            arr[i][j] = arr[j][i];
            arr[j][i] = temp;

        }
    }
    // reeverse evry row
    for(int i=0;i<n;i++){
        int left = 0, right = n-1;
        while(left<right)
        {
            int temp = arr[i][left];
            arr[i][left] = arr[i][right];
            arr[i][right] = temp;
            right --; left ++;
        }
    }
    return arr;
}
}
```

### Set_Matrix_Zeroes.java

```
//package PLACEMENTS.striver.arrays.faq_medium;
/*Given an m x n integer matrix matrix, if an element is 0, set its entire row and column to 0. You must do it in
place.

Example 1:
Input: matrix = [[1,1,1],[1,0,1],[1,1,1]]

Output: [[1,0,1],[0,0,0],[1,0,1]]

Explanation:

Element at position (1,1) is 0, so set entire row 1 and column 1 to 0.

Example 2:
Input: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]

Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]

Explanation:

There are two zeroes: (0,0) and (0,3).

Row 0 → all elements become 0
Column 0 and column 3 → all elements become 0 */

    import java.util.*;

public class Set_Matrix_Zeroes {

    public static void main(String[] args) {

        Scanner s = new Scanner(System.in);

        int m = s.nextInt();
        int n = s.nextInt();

        int matrix[][] = new int[m][n];

        for(int i = 0; i < m; i++) {
            for(int j = 0; j < n; j++) {
                matrix[i][j] = s.nextInt();
            }
        }

        app1(matrix);

        System.out.println(Arrays.deepToString(matrix));
    }

    // Approach 1: Brute Force (Store zero positions)
    // Time: O(m*n + z*(m+n))
    // Space: O(z)
    // z = number of original zeroes
```

```
    public static void app1(int matrix[][]) {

        ArrayList<Integer> pos = new ArrayList<>();

        for(int i = 0; i < matrix.length; i++) {
            for(int j = 0; j < matrix[0].length; j++) {

                if(matrix[i][j] == 0) {

                    // Store row and column positions
                    pos.add(i);
                    pos.add(j);
                }
            }
        }

        zero_down(matrix, pos);
    }

    public static void zero_down(int matrix[][], ArrayList<Integer> pos) {

        for(int i = 0; i < pos.size(); i = i + 2) {

            // Make entire row zero
            for(int k = 0; k < matrix[0].length; k++) {
                matrix[pos.get(i)][k] = 0;
            }

            // Make entire column zero
            for(int k = 0; k < matrix.length; k++) {
                matrix[k][pos.get(i+1)] = 0;
            }
        }
    }
}
```

### spiral_matrix.java

```
/*Given an M * N matrix, print the elements in a clockwise spiral manner.

Return an array with the elements in the order of their appearance when printed in a spiral manner.

Example 1:
Input: matrix = [[1, 2, 3], [4 ,5 ,6], [7, 8, 9]]

Output: [1, 2, 3, 6, 9, 8, 7, 4, 5]

Explanation:

The elements in the spiral order are 1, 2, 3 -> 6, 9 -> 8, 7 -> 4, 5

Example 2:
Input: matrix = [[1, 2, 3, 4], [5, 6, 7, 8]]

Output: [1, 2, 3, 4, 8, 7, 6, 5]

Explanation:

The elements in the spiral order are 1, 2, 3, 4 -> 8, 7, 6, 5*/
import java.util.*;
class spiral_matrix {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int m = s.nextInt();
        int n = s.nextInt();
        int arr [][] = new int[m][n];
        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++) arr[i][j] = s.nextInt();
        }
        System.out.print(spiralOrder(arr));
    }
    public static List<Integer> spiralOrder(int[][] arr) {
        List<Integer> res = new ArrayList<>();
        int i=0,j=0; int r = arr.length; int c = arr[0].length;
            int r2=0, c2 = 0;
            int total = arr.length*(arr[0].length);
        while(res.size() <total){
            while(j<c){ // go left
                res.add(arr[i][j]); j++;
                System.out.println("res now : "+res);
            }
```

```
          j--; i++;
           //return res;
          while(i<r){ // go down
               res.add(arr[i][j]); i++;
               System.out.println("res now : "+res);
           }
           i--; j--;
           //return res;
          while(j>=c2 &&  res.size() < total){ // go left
               res.add(arr[i][j]); j--;
               System.out.println("res now : "+res);
           }
           j++; i--;
           //return res;
           while(i>r2 &&  res.size() < total){ // go right
               res.add(arr[i][j]); i--;
               System.out.println("res now : "+res);
           }
           i++; j++;
           //return res;
           c--; r--; c2++; r2++;
       }
       return res;
   }}
   // nen miss ayindi while lopala res.size rayadam , dani valla problem ayithadi ani nak teludu
```

# striver / basicmaths

## gcd.java

```
/*You are given two integers n1 and n2. You need find the Greatest Common Divisor (GCD) of the two given numbers.
Return the GCD of the two numbers.



The Greatest Common Divisor (GCD) of two integers is the largest positive integer that divides both of the integers.


Example 1

Input: n1 = 4, n2 = 6

Output: 2

Explanation: Divisors of n1 = 1, 2, 4, Divisors of n2 = 1, 2, 3, 6

Greatest Common divisor = 2.

Example 2

Input: n1 = 9, n2 = 8

Output: 1

Explanation: Divisors of n1 = 1, 3, 9 Divisors of n2 = 1, 2, 4, 8.

Greatest Common divisor = 1. */
import java.util.*;
class gcd{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n1= s.nextInt();
        int n2 = s.nextInt();
        approch_2(n1,n2);
        approch_3(n1,n2);
        int ans = 1;
        for(int i=2;i<=Math.min(n1,n2);i++){
            if((n1%i==0) && (n2%i==0)){
                ans = i;
            }
            //System.out.println(i);
        }
        System.out.println(ans);

    }
public static void approch_2(int n1,int n2){
    for(int i=Math.min(n1,n2);i>=1;i--){
        if(n1%i == 0 && n2%i == 0){
            System.out.println(i);
            break;
        }
    }
}
public static void approch_3(int a,int b){
    while (a>=0 && b >=0){
        if (a==0){
            System.out.println(b);
            break;
        }
        else if(b==0){
        System.out.println(a);
        break;}
        else if(a>b){
            a = a%b;
        }
        else if(b>a){
            b = b%a ;
        }
    }
}
}
```

### primeno.java

```
/*You are given an integer n. You need to check if the number is prime or not. Return true if it is a prime number,
otherwise return false.



A prime number is a number which has no divisors except 1 and itself.


Example 1

Input: n = 5

Output: true

Explanation: The only divisors of 5 are 1 and 5 , So the number 5 is prime.

Example 2

Input: n = 8

Output: false

Explanation: The divisors of 8 are 1, 2, 4, 8, thus it is not a prime number. */
import java.util.*;
public class primeno{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        boolean res = false;
        for(int i=2;i<n;i++){
            // i * i <= n best
            if(n%i ==0){
                //System.out.println("%d is not a prime number",n);
                //ln rayaddu annta
                System.out.printf("%d is not a prime number",n);
                res = true;
                break;
            }
        }
        if(res == false)
        System.out.printf("%d is a prime number",n);
    }
}
// time complexity --> O(n)
//space complexity --> O(1)
//when changed to root n
// time ---> O(root(n)) and space --> O(1)
// edge cases : n = 1,
```

# striver / binary_search / 2d array

### Find_Row_with_Maximum_1s.java

```
/*707. Find row with maximum 1's
Given a non-empty grid mat consisting of only 0s and 1s, where all the rows are sorted in ascending order, find the
index of the row with the maximum number of ones.
If two rows have the same number of ones, consider the one with a smaller index. If no 1 exists in the matrix,
return -1.
Example 1:
Input : mat = [ [1, 1, 1], [0, 0, 1], [0, 0, 0] ]
Output: 0
Explanation: The row with the maximum number of ones is 0 (0 - indexed).
Example 2:
Input: mat = [ [0, 0], [0, 0] ]
Output: -1
Explanation: The matrix does not contain any 1. So, -1 is the answer.*/
//package PLACEMENTS.striver.binary_search.2d array;
import java.util.*;

public class Find_Row_with_Maximum_1s {
    public static void main(String args[])    {
        Scanner s = new Scanner(System.in);
        int m= s.nextInt();
        int n = s.nextInt();
        int arr [][] = new int [m][n];
        for (int i=0;i<m;i++) {
            for (int j=0;j<n;j++){
                arr[i][j] = s.nextInt();
            }
    }
    System.out.println(app1(arr));
    System.out.println(app2(arr));
}
public static int app1(int arr[][]){
    int best_ones = 0; // ee intializations chala chala imp
    int best_row = -1;
    for(int i=0;i<arr.length;i++){
        int res = calc_no_of_ones(arr[i]);
        if(res> best_ones){
            best_ones = res;
            best_row = i;
        }
    }
    return best_row;
}
public static int calc_no_of_ones(int arr[]){
    int low = 0, high = arr.length-1;
    int res = -1;
    while(low <= high) {
        int mid = low + (high - low )/2;
        if(arr[mid]== 1){
            res = mid;
            high = mid-1;
        }
        else { low = mid +1;}
    }
    if(res == -1) return 0;
    return arr.length - res; // no of ones
    }
    public static int app2(int arr[][]){
        int i=0, j = arr[0].length-1; // start with top right
        int best_row = -1;
        while(i<= arr.length-1 && j>= 0){
            if(arr[i][j] == 1){
                best_row = i;
                j--;
            }
            else{
                i++;
            }
        }
       return best_row;
    }
}
```

# striver / binary_search / fundamentals

### bs.java

```
// pre fs lo unntadi chuko poooo
```

### lowerbound.java

```
/*Given a sorted array of nums and an integer x, write a program to find the lower bound of x.

The lower bound algorithm finds the first and smallest index in a sorted array where the value at that index is
greater than or equal to a given key i.e. x.

If no such index is found, return the size of the array.

Example 1:
Input : nums= [1,2,2,3], x = 2

Output:1

Explanation:

Index 1 is the smallest index such that arr[1] >= x.

Example 2:
Input : nums= [3,5,8,15,19], x = 9

Output: 3

Explanation:

Index 3 is the smallest index such that arr[3] >= x*/
import java.util.*;
public class lowerbound{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int target = s.nextInt();
        System.out.println(app1(arr,n,target));
    }
    public static int app1(int arr[],int n,int target){
        int low = 0, high = n-1;
        int res = n;
        while(low<=high){
            int mid = low + (high-low)/2;
            if(arr[mid]>= target){ // as it is question anntey
                res = mid;
                high = mid -1; // heart of problem , as we need smallest index so it will search for that min
possible index and once it fails it returns the smallest index
            }
            else{
                low = mid +1;
            }
            // bs laga 3 cases avasram ledu ee design lo


        }
        return res;

    }
}
//https://claude.ai/share/13f524a4-7818-466d-9363-04f62daec3b7
```

### Search_in_a_2D_Matrix.java

```
/*77. Search in a 2D Matrix

48


Companies
Given a 2-D array mat where the elements of each row are sorted in non-decreasing order, and the first element of a
row is greater than the last element of the previous row (if it exists), and an integer target, determine if the
target exists in the given mat or not.

Example 1:
Input: mat = [ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12] ], target = 8
```

```
Output: True

Explanation: The target = 8 exists in the 'mat' at index (1, 3).

Example 2:
Input: mat = [ [1, 2, 4], [6, 7, 8], [9, 10, 34] ], target = 78

Output: False

Explanation: The target = 78 does not exist in the 'mat'. Therefore in the output, we see 'false'.*/
import java.util.*;
public class Search_in_a_2D_Matrix {

}
```

### upperbound.java

```
/*Given a sorted array of nums and an integer x, write a program to find the upper bound of x.

The upper bound of x is defined as the smallest index i such that nums[i] > x.

If no such index is found, return the size of the array.

Example 1:
Input : n= 4, nums = [1,2,2,3], x = 2

Output:3

Explanation:

Index 3 is the smallest index such that arr[3] > x.

Example 2:
Input : n = 5, nums = [3,5,8,15,19], x = 9

Output: 3

Explanation:

Index 3 is the smallest index such that arr[3] > x.*/
import java.util.*;
public class upperbound{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int target = s.nextInt();
        System.out.println(app1(arr,n,target));
    }
    public static int app1(int arr[],int n,int target){
        int low = 0, high = n-1;
        int res = n;
        while(low<=high){
            int mid = low + (high-low)/2;
            if(arr[mid]>target){
                res = mid;
                high = mid-1;
            }
            else{
           low = mid +1;
            }
            // bs laga 3 cases avasram ledu ee design lo
        }
        return res;
    }
}
```

# striver / binary_search / logic_building

### Count_Occurrences_in_a_Sorted_Array.java

```
/*You are given a sorted array of integers arr and an integer target. Your task is to determine how many times
target appears in arr.

Return the count of occurrences of target in the array.

Example 1:
Input: arr = [0, 0, 1, 1, 1, 2, 3], target = 1

Output: 3

Explanation: The number 1 appears 3 times in the array.

Example 2:
Input: arr = [5, 5, 5, 5, 5, 5], target = 5

Output: 6

Explanation: All elements in the array are 5, so the target appears 6 times. */
import java.util.*;
public class Count_Occurrences_in_a_Sorted_Array{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int target = s.nextInt();
        System.out.print(app1(arr, target));
    }
    public static int app1(int arr[], int target){
        int low = 0, high = arr.length - 1;
        while(low <= high ){
            int mid = low + (high - low )/2;
            if()
        }
    }
}
```

### Find_minimum_in_Rotated_Sorted_Array.java

```
//Find_minimum_in_Rotated_Sorted_Array
/*Given an integer array nums of size N, sorted in ascending order with distinct values, and then rotated an unknown
number of times (between 1 and N), find the minimum element in the array.

Example 1:
Input : nums = [4, 5, 6, 7, 0, 1, 2, 3]

Output: 0

Explanation: Here, the element 0 is the minimum element in the array.

Example 2:
Input : nums = [3, 4, 5, 1, 2]

Output: 1

Explanation:Here, the element 1 is the minimum element in the array. */
import java.util.*;
public class Find_minimum_in_Rotated_Sorted_Array
{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = s.nextInt();
        }
        System.out.println(app1(arr,n));
        System.out.print(app2(arr,n));
    }
    public static int app1(int arr[],int n){
        int low = 0, high = n-1;
        int res = Integer.MAX_VALUE;
        while(low<=high){
            int mid = low + (high -low)/2;
            if(arr[mid]>=arr[low]){
                res = Math.min(res,arr[low]);
```

```
                low = mid +1;
            }
            else{
                res = Math.min(res,arr[mid]);
                high = mid -1;
            }
        }
        return res;
    }
     public static int app2(int arr[],int n){
        int low = 0, high = n-1;
        int res = Integer.MAX_VALUE;
        while(low<=high){
            if(arr[low]<= arr[high]) return Math.min(res,arr[low]);
            int mid = low + (high -low)/2;
            if(arr[mid]>=arr[low]){
                res = Math.min(res,arr[low]);
                low = mid +1;
            }
            else{
                res = Math.min(res,arr[mid]);
                high = mid -1;
            }
        }
        return res;
    }
}
```

### Find_Number_of_Times_Array_is_Rotated.java

```
/*Given an integer array nums of size n, sorted in ascending order with distinct values.
The array has been right rotated an unknown number of times, between 0 and n-1 (including).
Determine the number of rotations performed on the array.

Example 1:
Input : nums = [4, 5, 6, 7, 0, 1, 2, 3]

Output: 4

Explanation: The original array should be [0, 1, 2, 3, 4, 5, 6, 7]. So, we can notice that the array has been
rotated 4 times.

Example 2:
Input: nums = [3, 4, 5, 1, 2]

Output: 3

Explanation: The original array should be [1, 2, 3, 4, 5]. So, we can notice that the array has been rotated 3
times.*/
import java.util.*;
public class Find_Number_of_Times_Array_is_Rotated{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = s.nextInt();
        }
        System.out.println(app1(arr,n));

    }
    public static int app1(int arr[],int n){
        int low = 0, high = n-1;
        int min_value= Integer.MAX_VALUE;
        int min_index = -1;
        while(low<=high){
            int mid = low + (high -low)/2;
            if(arr[mid]>=arr[low]){ // left sorted
                if(min_value> arr[low]){ // ofc min element is low
                    min_value = arr[low];
                    min_index = low;
                }
                low = mid +1;
            }
            else{
              if(min_value >arr[mid]){
                    min_value = arr[mid];
                    min_index = mid;
                }
                high = mid -1;
            }
```

```
        }
        return min_index;
    }
}
//https://claude.ai/share/cd137ef9-db32-4f1f-b074-5d1dc24d7288
```

### First_and_last_occurrence.java

```
/*Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given
target value. If the target is not found in the array, return [-1, -1].

Example 1:
Input: nums = [5, 7, 7, 8, 8, 10], target = 8

Output: [3, 4]

Explanation:The target is 8, and it appears in the array at indices 3 and 4, so the output is [3,4]

Example 2:
Input: nums = [5, 7, 7, 8, 8, 10], target = 6

Output: [-1, -1]

Expalantion: The target is 6, which is not present in the array. Therefore, the output is [-1, -1].

Now Your Turn!
Pick the correct output for the given input
Input: nums = [5, 7, 7, 8, 8, 10], target = 5 */
import java.util.*;
public class First_and_last_occurrence{
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++) arr[i] =s.nextInt();
        int target = s.nextInt();
        System.out.print(Arrays.toString(app1(arr,n,target)));
        System.out.print(Arrays.toString(app2(arr,n,target)));
    }
    public static int [] app1(int arr[],int n,int target){
        int res[] = new int [2];
        int i=0, j=n-1;
        while(i<=j){
        if(arr[i] < target && arr[j]>target){
            i++; j--;
        }
        if(arr[i]== target && arr[j] == target) {
            res[0] = i; res[1] = j;
            return res;
        }
        if((arr[i] > target || arr[j]<target)){
            res[0] = -1;res[1] = -1;
            return res;
        }
        //only i increases
        if(arr[i]<target) i++;
        if(arr[j]>target) j--;
    }
    return new int[]{-1, -1};
    }
    // Approach 2: two binary searches. Time O(log n), space O(1)
    public static int[] app2(int arr[], int n, int target) {
        return new int[]{first(arr, n, target), last(arr, n, target)};
    }

    // Binary search 1: first occurrence, or -1 if not present
    static int first(int arr[], int n, int target) {
        int low = 0, high = n - 1;
        int result = -1;                      // stays -1 if target is missing

        while (low <= high) {
            int mid = low + (high - low) / 2; // avoids overflow

            if (arr[mid] == target) {
                result = mid;                 // save as possible answer
                high = mid - 1;               // keep looking on the left
            } else if (arr[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
```

```
        }
        return result;
    }

    // Binary search 2: last occurrence, or -1 if not present
    static int last(int arr[], int n, int target) {
        int low = 0, high = n - 1;
        int result = -1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (arr[mid] == target) {
                result = mid;                // save as possible answer
                low = mid + 1;               // keep looking on the right
            } else if (arr[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return result;
    }
}
```

### Floor_and_Ceil_in_Sorted_Array.java

```
/*Given a sorted array nums and an integer x. Find the floor and ceil of x in nums. The floor of x is the largest
element in the array which is smaller than or equal to x. The ceiling of x is the smallest element in the array
greater than or equal to x. If no floor or ceil exists, output -1.

Example 1:
Input : nums =[3, 4, 4, 7, 8, 10], x= 5

Output: 4 7

Explanation: The floor of 5 in the array is 4, and the ceiling of 5 in the array is 7.

Example 2:
Input : nums =[3, 4, 4, 7, 8, 10], x= 8

Output: 8 8

Explanation: The floor of 8 in the array is 8, and the ceiling of 8 in the array is also 8.

Example 3:
Input : nums = [2, 4, 6, 8, 10, 12, 14], x= 1

Output:
[-1, 2]
Still unsure what the problem is asking ?

Let’s go through a few more examples, step by step, to make it clearer.

Constraints:
  1 <= nums.length <= 105
  0 < nums[i], x < 105
  nums is sorted in ascending order. */
import java.util.*;
public class  Floor_and_Ceil_in_Sorted_Array{
  public static void main(String args[]){
    Scanner s = new Scanner(System.in);
    int n = s.nextInt();
    int arr [] = new int[n];
    for (int i=0;i<n;i++) arr[i] = s.nextInt();
    int target = s.nextInt();
    System.out.print(Arrays.toString((bs(arr,n,target))));
  }
  public static int [] bs(int arr[], int n, int target){
    int res [] = new int[2];
    int low = 0, high = arr.length-1;
    while(low<= high){
    int mid = low + (high - low )/2;
    if(arr[mid] == target){
      res[0] = target; res[1] = target;
      return res;
    }
    else if(arr[mid]>target){
      high = mid -1;
    }
    else {
```

```
      low = mid +1;
    }
  }
    // security guards
    if(high <0){ // see carefully high emo 0 vaipu vastunndi and low emo n vaipu vastunndi
      res[0] = -1;
    }
    else{
      res[0] = arr[high];
    }
    // low kosam kuda
    if(low > n-1){
      res[1] = -1;
    }
    else{
      res[1] = arr[low];
    }
  return res;

}
}
//https://claude.ai/share/0e375aca-e610-4a8e-83bf-c5eb9f4ef8bb
//dsa 11
```

### Kth_Missing_Positive_Number.java

```
/*Given a sorted array of unique positive integers arr, your task is to return the kth missing positive number that
is not present in arr.

The array is guaranteed to be strictly increasing, and the missing numbers are those positive integers that do not
appear in arr but would appear in a full sequence starting from 1.

Example 1:
Input: arr = [3, 5, 7, 10], k = 6

Output: 9

Explanation:

The missing numbers are [1, 2, 4, 6, 8, 9, 11, ...]. The 6th missing number is 9.

Example 2:
Input: arr = [1, 4, 6, 8, 9], k = 3

Output: 5

Explanation:

The missing numbers are [2, 3, 5, 7, 10, ...]. The 3rd missing number is 5.
Constraints:
1 <= arr.length <= 1000
1 <= arr[i] <= 1000
1 <= k <= 1000
arr[i] < arr[j] for all 1 <= i < j <= arr.length*/
import java.util.*;
public class Kth_Missing_Positive_Number {
    public static void main(String args[])    {
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int [n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int k = s.nextInt();
        System.out.println(app1(arr,k));
    }
    public static int app1(int arr[],int k){
        int x = 1; int count = 0;
        while(true){
            if(!is_present(arr,x)){
                count++;
            }
            if(count == k){
                return x;
            }
            x++;
            }
    }
    public static boolean is_present(int arr[],int x){
        int low = 0, high = arr.length-1;
        while(low <= high ){
            int mid = low + (high -low)/2;
            if(arr[mid] == x) return true;
```

```
            else if(arr[mid]>x) high = mid -1;
            else low = mid +1;
        }
        return false;
    }
}
```

### Search_in_rotated_sorted_array_I.java

```
/*Given an integer array nums, sorted in ascending order (with distinct values) and a target value k. The array is
rotated at some pivot point that is unknown. Find the index at which k is present and if k is not present return -1.
Example 1:
Input : nums = [4, 5, 6, 7, 0, 1, 2], k = 0
Output: 4
Explanation: Here, the target is 0. We can see that 0 is present in the given rotated sorted array, nums. Thus, we
get output as 4, which is the index at which 0 is present in the array.
Example 2:
Input: nums = [4, 5, 6, 7, 0, 1, 2], k = 3

Output: -1

Explanation: Here, the target is 3. Since 3 is not present in the given rotated sorted array. Thus, we get the
output as -1.Given an integer array nums, sorted in ascending order (with distinct values) and a target value k. The
array is rotated at some pivot point that is unknown. Find the index at which k is present and if k is not present
return -1.

Example 1:
Input : nums = [4, 5, 6, 7, 0, 1, 2], k = 0

Output: 4

Explanation: Here, the target is 0. We can see that 0 is present in the given rotated sorted array, nums. Thus, we
get output as 4, which is the index at which 0 is present in the array.

Example 2:
Input: nums = [4, 5, 6, 7, 0, 1, 2], k = 3

Output: -1

Explanation: Here, the target is 3. Since 3 is not present in the given rotated sorted array. Thus, we get the
output as -1. */
//Search_in_rotated_sorted_array-I
import java.util.*;
public class Search_in_rotated_sorted_array_I{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr[] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int target = s.nextInt();
        System.out.println(shrewwni(arr,n,target));

    }
    public static int shrewwni(int arr[], int n,int target){
        int low = 0, high = n-1;
        while(low<= high){
            int mid = low + (high - low)/2;
            if(arr[mid] == target) return mid;
            else if(arr[low] <=arr[mid]){ // left sorted
                if(arr[low]<=target && arr[mid]>=target){
                     high = mid -1;
                }
                else {
                    low = mid +1;
                }
            }
            else{ // right sorted
                if(arr[high]>=target && arr[mid]<=target){
                    low = mid +1;
                }
                else{
                    high = mid -1;
                }
            }
        }
        return -1;
    }
}
```

### Search_in_Rotated_Sorted_Array_II.java

```
/*Given an integer array nums, sorted in ascending order (may contain duplicate values) and a target value k. Now
the array is rotated at some pivot point unknown to you. Return True if k is present and otherwise, return False.

Example 1:
Input : nums = [7, 8, 1, 2, 3, 3, 3, 4, 5, 6], k = 3

Output: True

Explanation: The element 3 is present in the array. So, the answer is True.

Example 2:
Input : nums = [7, 8, 1, 2, 3, 3, 3, 4, 5, 6], k = 10

Output: False

Explanation:The element 10 is not present in the array. So, the answer is False.*/
import java.util.*;
public class Search_in_Rotated_Sorted_Array_II{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int target = s.nextInt();
        System.out.println(app1(arr,n,target));
    }
    public static boolean app1(int arr[], int n ,int target){
        int low = 0 , high = n-1;
        while(low <=high){
            int mid = low + (high-low )/2;
            if(arr[mid] == target) return true;
            else if(arr[mid] == arr[low] && arr[mid] == arr[high]) { // heart of the problem
                low++; high--;
            }
            else if(arr[mid] >= arr[low]){ // left is sorted
                if(arr[mid]>= target && arr[low]<=target){
                    high = mid -1;

                }
                else {
                    low = mid +1;
                }
            }
            else{
                if(arr[high]>=target && arr[mid]<= target){
                    low = mid +1;
                }
                else {
                    high = mid -1;
                }
            }
        }
        return false ;
    }
}
//https://claude.ai/share/9e58da9f-06b9-459b-93f6-18dd288fe32c
```

### search_insert_pos.java

```
/*Given a sorted array of nums consisting of distinct integers and a target value, return the index if the target is
found. If not, return the index where it would be if it were inserted in order.

Example 1:
Input: nums = [1, 3, 5, 6], target = 5

Output: 2

Explanation: The target value 5 is found at index 2 in the sorted array. Hence, the function returns 2.

Example 2:
Input: nums = [1, 3, 5, 6], target = 2

Output: 1

Explanation: The target value 2 is not found in the array. However, it should be inserted at index 1 to maintain the
sorted order of the array.*/
import java.util.*;
public class search_insert_pos{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
```

```
        int n = s.nextInt();
        int arr[] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int target = s.nextInt();
        System.out.println(bs(arr,n,target));
    }
    public static int bs(int arr[], int n , int target){
        int low = 0, high = n-1;
        while(low<= high){
            int mid = low + (high - low )/2;
            if(arr[mid] == target){
                return mid;
            }
            else if(arr[mid]<target){
                low = mid +1;
            }
            else {
                high = mid -1;
            }
        }
            return low; // the heart of the problem
    }
}
//https://claude.ai/share/0e375aca-e610-4a8e-83bf-c5eb9f4ef8bb
//dsa 11
```

### Single_Element_in_Sorted_Array.java

```
/*Given an array nums sorted in non-decreasing order. Every number in the array except one appears twice. Find the
single number in the array.

Example 1:
Input :nums = [1, 1, 2, 2, 3, 3, 4, 5, 5, 6, 6]

Output:4

Explanation: Only the number 4 appears once in the array.

Example 2:
Input : nums = [1, 1, 3, 5, 5]

Output:3

Explanation: Only the number 3 appears once in the array. */
import java.util.*;
public class Single_Element_in_Sorted_Array{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for (int i=0;i<n;i++) arr[i] = s.nextInt();
        System.out.println(app1(arr)); // idhi nen rasina
        // migtavi given claude to check other alternatives
    }
    public static int app1(int arr[]){
        int low = 0, high = arr.length-1;
        while(low <= high ){
            int mid = low + (high - low )/2;
            if(low == high ) // or high - low = 0
            { return arr[low]; // high or mid kuda return cheayvachu
            }
            else if(arr[mid] == arr[mid-1]){ // paired with left side
                int left_length = (mid -1) - low ; // zero based indexing kada
                if(left_length % 2 ==0) {
                    low = mid +1; // heart of the problem
                }
                else{
                    high = mid -2;
                }
            }
            else if (arr[mid] == arr[mid+1]){ // paired with right half
                int left_length = mid -low;
                if(left_length%2 == 0) {
                    low = mid +2;
                }
                else{
                    high = mid -1;
                }
            }
            else {return arr[mid];}
        }
```

```
        return -1 ; // never excutess
    }
    public static int linearScan(int[] arr) {
    int n = arr.length;
    if (n == 1) return arr[0]; // to stop array out of bounds
    if (arr[0] != arr[1]) return arr[0]; // 1st element special check
    if (arr[n - 1] != arr[n - 2]) return arr[n - 1]; // last element special check
    for (int i = 1; i < n - 1; i++)
        if (arr[i] != arr[i - 1] && arr[i] != arr[i + 1]) return arr[i]; // other checks
    return -1;
}
public static int evenJump(int[] arr) { // idhi naa idea
    int n = arr.length;
    for (int i = 0; i < n - 1; i += 2) // n-1 varake povali
        if (arr[i] != arr[i + 1]) return arr[i];
    return arr[n - 1]; // last element ee answer iga
}
public static int bruteMap(int[] arr) { // ofc idhi manam ila cheyam , just to remind you the approch of hashmap
    HashMap<Integer, Integer> map = new HashMap<>();
    for (int x : arr) map.put(x, map.getOrDefault(x, 0) + 1);
    for (Map.Entry<Integer, Integer> e : map.entrySet())
        if (e.getValue() == 1) return e.getKey();
    return -1;
}
}
//https://claude.ai/share/d0b9c1d1-28fa-4fca-abab-2459d1e7b26b
```

# striver / binary_search / on_answers

### Capacity_to_Ship_Packages_Within_D_Days.java

```
/*You are given an array weights where weights[i] represents the weight of the i-th package on a conveyor belt. All
the packages must be shipped in the order given from one port to another within days days.

Each day, the ship can carry a contiguous sequence of packages, as long as the total weight does not exceed its
maximum capacity.

Your task is to find the minimum possible capacity of the ship so that all packages can be shipped within the given
number of days.

Example 1:
Input: weights = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], days = 5

Output: 15

Explanation:

Minimum ship capacity = 15. One way to ship in 5 days:

Day 1: 1 + 2 + 3 + 4 + 5 = 15
Day 2: 6 + 7 = 13
Day 3: 8
Day 4: 9
Day 5: 10
No day exceeds capacity 15 and all packages are shipped in order in 5 days.

Example 2:
Input: weights = [3, 2, 2, 4, 1, 4], days = 3

Output: 6

Explanation:

One possible division with capacity 6:

Day 1: 3 + 2 = 5
Day 2: 2 + 4 = 6
Day 3: 1 + 4 = 5
All packages shipped in order within 3 days.
Constraints:
1 <= days <= weights.length <= 5 * 10⁴
1 <= weights[i] <= 500*/
import java.util.*;
public class Capacity_to_Ship_Packages_Within_D_Days {
public static void main(String args[]) {
    Scanner s = new Scanner (System.in);
    int n = s.nextInt();
    int arr [] = new int [n];
    for(int i=0;i<n;i++) arr[i]= s.nextInt();
    int days = s.nextInt();
    System.out.println(app1(arr,days));
}
public static int app1(int arr [], int days){
    int low = Integer.MIN_VALUE,high = 0;
    for(int x :arr) {
        low = Math.max(low,x); // heart of the problem
        high += x;
    }
    int res = -1;
    while(low <= high ){
        int mid = low + (high - low)/2;
        if(calc_days(arr,mid)<= days) { //  with in days kada
            res = mid; high = mid - 1;
        }
        else { low = mid +1;}
    }
    return res;
}
public static int calc_days(int arr[], int capacity){
    int current_capacity = capacity;
    int days = 1; // day one nuuchi start avvali enduku anntey aadhi ofc kabatti
    for (int i=0;i<arr.length;i++){
        if(arr[i]<= current_capacity){
            current_capacity-= arr[i];
        }
        else{
            current_capacity = capacity; // refuel cheyali kada ofc
            days ++; // oka day ayipoindi
            current_capacity -= arr[i]; // refuel chesam bane unndi kani, current arr[i] pasitit endi , adhi kotha
```

```
day capacity lo add kavali kada , andukey add chestunnam ikkada
            // idey nen marchipoyina (sad)
        }
    }
    return days;
}
}
//https://claude.ai/share/cde1a74c-274d-48f3-ba5d-579fd1cdd5bd
```

### Find_Nth_Root_of_a_Number.java

```
/*Given two numbers N and M, find the Nth root of M. The Nth root of a number M is defined as a number X such that
when X is raised to the power of N, it equals M. If the Nth root is not an integer, return -1.

Example 1:
Input: N = 3, M = 27

Output: 3

Explanation: The cube root of 27 is equal to 3.

Example 2:
Input: N = 4, M = 69

Output:-1

Explanation: The 4th root of 69 does not exist. So, the answer is -1.

Example 3:
Input: N = 4, M = 81

Output:
3 */
import java.util.*;
public class Find_Nth_Root_of_a_Number{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int m = s.nextInt();
        System.out.println(app1(n,m));
        System.out.println(app2(n,m));
        System.out.println(app3(n,m));
    }
    public static int app1(double n,double m){
        double pow = Math.pow(m,1/n);
        System.out.print(pow );
        if((pow - Math.ceil(pow))==0){ // integer aa kada ani check chestunnam
            return (int)pow;
        }
        return -1;
    } // understand y it is failing
    public static int app2(int n,int m){
        int x = 0;
        while(true){
            if(Math.pow(x,n) == m) return x;
            else if(Math.pow(x,n)<m) x++;
            else return -1;
        }
    }
    public static int app3(int n,int m){
        int low = 0, high = m; // aa number ee high pedtam
        while(low<= high){
            int mid = low + (high - low)/2;
            if(Math.pow(mid,n) == m) return mid;
            else if(Math.pow(mid,n)<m) low = mid +1;
            else high = mid -1;
        }
        return -1;
    }
}
//claude :
/*import java.util.*;

public class Find_Nth_Root_of_a_Number {

    // 0 -> mid^n < m,  1 -> mid^n == m,  2 -> mid^n > m
    static int compare(int mid, int n, int m) {
        long ans = 1;
        for (int i = 0; i < n; i++) {
            ans *= mid;
            if (ans > m) return 2;   // early exit, so no overflow
```

```
        }
        return ans == m ? 1 : 0;
    }

    // Method 1: Brute force (linear search on the answer)
    static int bruteForce(int n, int m) {
        for (int x = 1; x <= m; x++) {
            int c = compare(x, n, m);
            if (c == 1) return x;    // exact root found
            if (c == 2) return -1;   // went past m, so no integer root
        }
        return -1;
    }

    // Method 2: Binary search on the answer (optimal, interview answer)
    static int binarySearch(int n, int m) {
        int low = 1, high = m;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            int c = compare(mid, n, m);
            if (c == 1) return mid;
            else if (c == 0) low = mid + 1;   // mid^n too small, go right
            else high = mid - 1;              // mid^n too big, go left
        }
        return -1;
    }

    // Method 3: Round and verify (math shortcut)
    static int roundAndVerify(int n, int m) {
        int r = (int) Math.round(Math.pow(m, 1.0 / n));  // candidate only
        return compare(r, n, m) == 1 ? r : -1;           // exact check
    }

    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int m = s.nextInt();
        System.out.println(bruteForce(n, m));
        System.out.println(binarySearch(n, m));
        System.out.println(roundAndVerify(n, m));
    }
} */
```

### Find_Square_Root_of_a_Number.java

```
/*Given a positive integer n. Find and return its square root. If n is not a perfect square, then return the floor
value of sqrt(n).

Example 1:
Input: n = 36

Output: 6

Explanation: 6 is the square root of 36.

Example 2:
Input: n = 28

Output: 5

Explanation: The square root of 28 is approximately 5.292. So, the floor value will be 5. */
import java.util.*;
public class Find_Square_Root_of_a_Number{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        System.out.print(app1(n)); // ofc bakwas
        System.out.println(app2(n)); // ofc optimal
        System.out.println(app3(n));
    }
    public static int  app1(int n ){
        return (int ) Math.floor(Math.sqrt(n)); // answer double lo vastadi
    }
    public static int app2(int n){
        int low = 0 , high = n;  // high machiga chuskoo
        int res = -1;
        while(low<=high){
        int mid = low + (high-low)/2;
        if((long)mid*mid<= n){
            res = mid;
           low = mid+1;
```

```
        }
        else{
            high = mid -1;
        }
        }
        return res;
    }
    public static int app3(int n){ // intha chinna visyam rayaniki chala time pattindi (sad)
        int i= 0;
        while((long)i*i<=n) i++;
        return i -1;
    }

}
// long anede endukoo see in clude
//https://claude.ai/share/4c2deaa2-0923-4d90-820b-b4954890f107
```

### Find_the_Smallest_Divisor.java

```
/*Given an array of integers nums and an integer limit as the threshold value, find the smallest positive integer
divisor such that upon dividing all the elements of the array by this divisor, the sum of the division results is
less than or equal to the threshold value.

After dividing each element by the chosen divisor, take the ceiling of the result (i.e., round up to the next whole
number).

Example 1:
Input: nums = [1, 2, 3, 4, 5], limit = 8

Output: 3

Explanation: We can get a sum of 15(1 + 2 + 3 + 4 + 5) if we choose 1 as a divisor.

The sum is 9(1 + 1 + 2 + 2 + 3) if we choose 2 as a divisor. Upon dividing all the elements of the array by 3, we
get 1,1,1,2,2 respectively. Now, their sum is equal to 7 <= 8 i.e. the threshold value. So, 3 is the minimum
possible answer.

Example 2:
Input: nums = [8,4,2,3], limit = 10

Output: 2

Explanation: If we choose 1, we get 17 as the sum. If we choose 2, we get 9 (4+2+1+2) <= 10 as the answer. So, 2 is
the answer.*/
import java.util.*;
public class Find_the_Smallest_Divisor {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n  = s.nextInt();
        int arr [] = new int [n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int limit = s.nextInt();
        System.out.println(app1(arr,limit));
        System.out.println(app2(arr,limit));
    }
    public static int app1(int arr[], int limit){
        double x = 1; // inorder to statify the divide condition exactly we need double not int
        while(true){
            if(check_divide(arr,x,limit)){
                return (int)x;
            }
            x++;
        }

    }
    public static boolean check_divide(int arr[], double x ,int limit){ // double kavali
        int sum = 0;
        for(int i=0;i<arr.length;i++){
            sum += Math.ceil(arr[i]/x);
            if(sum>limit) return false;

        }
        return true;
    }
    public static int app2(int arr[],int limit){
        int low = 1; // smallest divisor one ee kada
        int high = Integer.MIN_VALUE;
        for(int x : arr) high = Math.max(high,x);
        int res = -1;
        while(low <= high){
            int mid = low + (high-low)/2;
```

```
            if(check_divide(arr, mid, limit)){
                res = mid;
                high = mid - 1;
            }
            else{
                low = mid +1;
            }
        }
        return res;
    }
}
//claude code :
/*import java.util.*;
public class Find_the_Smallest_Divisor {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n  = s.nextInt();
        int arr [] = new int [n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int limit = s.nextInt();
        System.out.println(app1(arr,limit));
        System.out.println(app2(arr,limit)); // ADDED: call the optimal approach too
    }

    // ---------- Approach 1: Brute force ----------
    public static int app1(int arr[], int limit){
        if (limit < arr.length) return -1;   // ADDED: sum is always >= n, so no divisor can work
        // ee pattern nen notice cheyale
        int max = Integer.MIN_VALUE;
        for (int v : arr) max = Math.max(max, v);

        // CHANGED: double x -> int x, and while(true) -> bounded loop (1..max).
        // At d = max the sum is exactly n <= limit, so an answer always exists by then.
        for (int x = 1; x <= max; x++) {
            if (check_divide(arr, x, limit)) {
                return x;                    // CHANGED: no (int) cast needed now
            }
        }
        return -1;                           // ADDED: safety return, never reached after the guard
    }

    // ---------- Approach 2: Binary search on the answer ----------
    public static int app2(int arr[], int limit){
        if (limit < arr.length) return -1;   // ADDED: impossible case, as you reasoned

        int low = 1;                         // smallest divisor is 1
        int high = Integer.MIN_VALUE;
        for(int x : arr) high = Math.max(high,x); // high = max(nums): sum is n here, can't improve beyond
        int res = -1;
        while(low <= high){
            int mid = low + (high-low)/2;
            if(check_divide(arr, mid, limit)){
                res = mid;                   // mid works, but a smaller divisor might also work
                high = mid - 1;
            }
            else{
                low = mid + 1;               // mid fails, so we need a bigger divisor
            }
        }
        return res;
    }

    // ---------- Check function ----------
    // CHANGED: parameter double x -> int x
    public static boolean check_divide(int arr[], int x, int limit){
        long sum = 0;                        // CHANGED: int -> long to avoid overflow at small divisors
        for(int i=0;i<arr.length;i++){
            // CHANGED: Math.ceil(arr[i] / x) -> integer ceiling formula (a + d - 1) / d.
            // The (long) cast keeps a + d - 1 from overflowing int.
            sum += ((long) arr[i] + x - 1) / x; // ade formula rasina chudu paina
            if(sum > limit) return false;    // early exit, kept from your version
        }
        return true;
    }
} */
//https://claude.ai/share/a7553df5-77bf-4250-9e8e-f372246cf787
```

### Koko_Eating_Bananas.java

```
/*A monkey is given n piles of bananas, where the 'ith' pile has nums[i] bananas. An integer h represents the total
time in hours to eat all the bananas.
Each hour, the monkey chooses a non-empty pile of bananas and eats k bananas.
If the pile contains fewer than k bananas, the monkey eats all the bananas in that
pile and does not consume any more bananas in that hour. // chala imp line idhi enduku anntey high value ni
determine chesidi idhey

Determine the minimum number of bananas the monkey must eat per hour to finish all the bananas within h hours.

Example 1:
Input: n = 4, nums = [7, 15, 6, 3], h = 8

Output: 5

Explanation: If Koko eats 5 bananas/hr, he will take 2, 3, 2, and 1 hour to eat the piles accordingly. So, he will
take 8 hours to complete all the piles.

Example 2:
Input: n = 5, nums = [25, 12, 8, 14, 19], h = 5

Output: 25

Explanation: If Koko eats 25 bananas/hr, he will take 1, 1, 1, 1, and 1 hour to eat the piles accordingly. So, he
will take 5 hours to complete all the piles.

Example 3:
Input: n = 4, nums = [3, 7, 6, 11], h = 8

Output:
4*/
import java.util.*;
public class Koko_Eating_Bananas {
public static void main(String args[])    {
    Scanner s = new Scanner(System.in);
    int n = s.nextInt();
    int arr [] = new int[n];
    for(int i=0;i<n;i++) arr[i] = s.nextInt();
    int hours = s.nextInt();
    System.out.println(app1(arr,n,hours));
}
public static int app1(int arr[], int n,int hours){
    int low = 1;
    int high = Integer.MIN_VALUE;
    for(int x : arr) high = Math.max(high,x);
    int res = -1;
    while(low<= high){
        int mid = low + (high - low)/2;
        int req_hours = calc_hours(arr,n,mid);
        System.out.println(req_hours + " " + mid);
        if(req_hours<= hours) {
            res = mid;
            high = mid -1;
        }
        else{
            low = mid +1;
        }
    }
    return res;
}
public static int calc_hours(int arr[], int n,double speed){ // divison kosam
    int req_hours = 0;
    for(int i=0;i<n;i++){
        req_hours += Math.ceil(arr[i]/speed);
        System.out.println(req_hours);
    }
    return req_hours;
}
}
// already motu somasa chesinam kada
```

### Minimum_Days_to_Make_M_Bouquets.java

```
/*Given n roses and an array nums where nums[i] denotes that the 'ith' rose will bloom on the nums[i]th day, only
adjacent bloomed roses can be picked to make a bouquet. Exactly k adjacent bloomed roses are required to make a
single bouquet. Find the minimum number of days required to make at least m bouquets, each containing k roses.
Return -1 if it is not possible.

Example 1:
Input: n = 8, nums = [7, 7, 7, 7, 13, 11, 12, 7], m = 2, k = 3
```

```
Output: 12

Explanation: On the 12th the first 4 flowers and the last 3 flowers would have already bloomed. So, we can easily
make 2 bouquets, one with the first 3 and another with the last 3 flowers.

Example 2:
Input: n = 5, nums = [1, 10, 3, 10, 2], m = 3, k = 2

Output: -1

Explanation: If we want to make 3 bouquets of 2 flowers each, we need at least 6 flowers. But we are given only 5
flowers, so, we cannot make the bouquets.

Now Your Turn!
Pick the correct output for the given input
Input: n = 5, nums = [1, 10, 3, 10, 2], m = 3, k = 1


10

1

2

3
Still unsure what the problem is asking ?

Let’s go through a few more examples, step by step, to make it clearer.

Constraints:
 1 <= n <= 105
 1 <= nums[i] <= 109
 1 <= m <= 106
 1 <= k <= n
Advertisements


Hints
Hint 1
Hint 2
Frequently Occurring Doubts
How is bouquet feasibility checked efficiently?
Why use binary search to find the minimum days?
Interview Follow-up Questions
How would you handle non-adjacent bouquets?
What if the roses bloom asynchronously, with delays or dependencies?*/
import java.util.*;
public class Minimum_Days_to_Make_M_Bouquets {
    public static void main(String args[])    {
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        int m = s.nextInt();
        int k = s.nextInt();
        if ((long) m * k > n) {System.out.println(-1); return;}// ee line ne pettale , claude pettindi but antha imp
emm kadu ,
        // ee line lekapoyina vastadi output just waste of computations anthey
        System.out.println(app1(arr,n,m,k));
    }
    public static int app1(int arr[], int n,int m,int k){
        int low = Integer.MAX_VALUE;
        int high = Integer.MIN_VALUE;
        for(int x:arr) {
            low = Math.min(low,x);
            high = Math.max(high,x);
        }
        //System.out.println(low + " " + high);
        int res = -1;
        while(low<= high){
            int mid = low + (high-low)/2;
            //System.out.print(mid);
            if(calc_bouques(arr,n, k,mid)>= m){ // atleast kada
                res = mid;
                //System.out.print(res + " " + calc_bouques(arr,n, k,mid));
                high = mid -1;
            }
            else{
                low = mid +1;
            }
        }
        return res;
    }
```

```
    public static int calc_bouques(int arr[],int n,int k,int day){ // mid anntey aa day ni artham
        int i=0;
        int bouque_count = 0;
        while(i<n){
            if(arr[i]<= day) { // bloomed
                 int bouque = 0;
                /*while(i<n && i< i +k && arr[i]<= day ){ // mistake see in notes
                    bouque ++; i++;
                }*/
               for(int j=0;j<k;j++){ // excatly k times run avali anntey best option for loop ee kada
                 if(i<n && arr[i]<= day) {
                    bouque ++; i++;
                 }
               }
                if(bouque == k) bouque_count++;
            }
            else {
                i++;
            }
        }
        return bouque_count;
    }
}
//claude ichindi :
/*static int count(int[] a, int k, int day) {
    int run = 0, bouquets = 0;
    for (int x : a) {
        if (x <= day) {
            run++;
            if (run == k) { bouquets++; run = 0; }
        } else run = 0;
    }
    return bouquets;
} */
//https://claude.ai/share/3b34564c-81b7-4ebd-b17d-5fd30e1510fe
```

# striver / greedy / easy

### Assign_Cookies.java

```
//package PLACEMENTS.striver.greedy.easy;
/*Consider a scenario where a teacher wants to distribute cookies to students, with each student receiving at most
one cookie.

Given two arrays, student and cookie, the ith value in the Student array describes the minimum size of cookie that
the ith student can be assigned. The jth value in the Cookie array represents the size of the jth cookie. If
Cookie[j] >= Student[i], the jth cookie can be assigned to the ith student.

Maximize the number of students assigned with cookies and output the maximum number.

Example 1:
Input : student = [1, 2, 3] , cookie = [1, 1]

Output :1

Explanation : You have 3 students and 2 cookies.

The minimum size of cookies required for students are 1 , 2 ,3.

You have 2 cookies both of size 1, So you can assign the cookie only to student having minimum cookie size 1.

So your answer is 1.

Example 2:
Input : student = [1, 2] , cookie = [1, 2, 3]

Output : 2

Explanation : You have 2 students and 3 cookies.

The minimum size of cookies required for students are 1 , 2.

You have 3 cookies and their sizes are big enough to assign cookies to all students.

So your answer is 2.

Example 3:
Input : student = [4, 5, 1] , cookie = [6, 4, 2]

Output:
3 */
//chatgpt code like me tho generatate chesina
import java.util.*;

public class Assign_Cookies {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        int n = s.nextInt();
        int student[] = new int[n];
        for (int i = 0; i < n; i++) student[i] = s.nextInt();

        int m = s.nextInt();
        int cookie[] = new int[m];
        for (int i = 0; i < m; i++) cookie[i] = s.nextInt();

        System.out.println(app1(student, cookie, n, m));
    }

    // Approach 1: Sorting + Two Pointers
    // Time: O(n log n + m log m), Space: O(1) auxiliary
    public static int app1(int student[], int cookie[], int n, int m) {
        Arrays.sort(student);
        Arrays.sort(cookie);

        int i = 0, j = 0;

        while (i < n && j < m) {
            if (cookie[j] >= student[i]) {
                i++;
            }
            j++;
        }

        return i;
    }
}
```

### Fractional_Knapsack.java

```
import java.util.*;

public class Fractional_Knapsack {

    public static void main(String[] args) {

        Scanner s = new Scanner(System.in);

        int n = s.nextInt();

        int val[] = new int[n];
        int wt[] = new int[n];

        for(int i = 0; i < n; i++) {
            val[i] = s.nextInt();
        }

        for(int i = 0; i < n; i++) {
            wt[i] = s.nextInt();
        }

        long cap = s.nextLong();

        System.out.println(app1(val, wt, n, cap));
    }

    // Approach 1: Sorting using value/weight ratio
    public static double app1(int val[], int wt[], int n, long cap) {

        double ratio[] = new double[n];

        for(int i = 0; i < n; i++) {
            ratio[i] = val[i] / (double)wt[i];
        }

        // Sort in descending order
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n - 1; j++) {

                if(ratio[j] < ratio[j+1]) {

                    double temp = ratio[j];
                    ratio[j] = ratio[j+1];
                    ratio[j+1] = temp;

                    int temp2 = val[j];
                    val[j] = val[j+1];
                    val[j+1] = temp2;

                    int temp3 = wt[j];
                    wt[j] = wt[j+1];
                    wt[j+1] = temp3;
                }
            }
        }

        double res = 0;

        for(int i = 0; i < n; i++) {

            if(cap >= wt[i]) {
                cap -= wt[i];
                res += val[i];
            }
            else {
                res += ratio[i] * cap;
                return res;
            }
        }

        return res;
    }
}
```

### Jump_Game_1.java

```
/*Given an array of integers nums, each element in the array represents the maximum jump length at that position.
Initially starting at the first index of the array, determine if it is possible to reach the last index. Return true
if the last index can be reached, otherwise return false.

Example 1:
Input : [2, 3, 1, 1, 4]

Output : true

Explanation : We can simply take Jump of 1 step at each index to reach the last index.

Example 2:
Input : [3, 2, 1, 0, 4]

Output : false

Explanation : No matter how you make jumps you will always reach the third index (0 base) of the array.

The maximum jump of index three is 0, So you can never reach the last index of array.

Example 3:
Input : [5, 3, 2, 1, 0]

Output:
true
Still unsure what the problem is asking ?

Let’s go through a few more examples, step by step, to make it clearer.

Constraints:
1 <= nums.length <= 104
0 <= nums[i] <= 105*/
import java.util.*;

public class Jump_Game_1{

    public static void main(String[] args) {

        Scanner s = new Scanner(System.in);

        int n = s.nextInt();

        int arr[] = new int[n];

        for(int i = 0; i < n; i++) {
            arr[i] = s.nextInt();
        }

        System.out.println(app1(arr, n));
    }

    // Approach 1: Greedy
    // Time: O(n), Space: O(1)

    public static boolean app1(int arr[], int n) {

        int maxReach = 0;

        for(int i = 0; i < n; i++) {

            // Check whether current index is reachable
            if(i > maxReach) { // zero block chesindi ani artham
                return false;
            }

            // Update maximum reachable index
            maxReach = Math.max(maxReach, i + arr[i]);

            // If last index is reachable
            if(maxReach >= n - 1) {
                return true;
            }
        }

        return true;
    }
}
```

### Lemonade_Change.java

```
/*Each lemonade at a booth sells for $5. Consumers are lining up to place individual orders, following the billing
order. Every consumer will purchase a single lemonade and may pay with a $5, $10, or $20 bill. Each customer must
receive the appropriate change so that the net transaction is $5. Initially, there is no change available.

Determine if it is possible to provide the correct change to every customer. Return true if the correct change can
be given to every customer, and false otherwise.

Given an integer array bills, where bills[i] is the bill the ith customer pays, return true if the correct change
can be given to every customer, and false otherwise.

Example 1:
Input : bills = [5, 5, 10, 5, 20]

Output : true

Explanation : Initially we have $0 available for change.

From first two customers, we will collect two $5 bills in order. After the first two customers we have two $5 bills
available with us for change.

From the third customer , we collect bill of $10 and give back $5. After third customer we have one $5 and one $10
bill available with us for change.

From fourth customer , we collect $5 bill. After fourth customer we have two $5 and one $10 bills available with us
for change if required.

From fifth customer , we collect bill of $20 and give back $15 (one $10 + one $5 bill).

Since all the customers did receive the change correctly , so we return true.

Example 2:
Input : bills = [5, 5, 10, 10, 20]

Output : false

Explanation : From first two customers, we will collect two $5 bills in order. After the first two customers we have
two $5 bills available with us for change.

From third customer , we collect $10 and give back $5. After the third customer we have one $5 and one $10 bill
available with us for change.

From fourth customer , we collect $10 and give back $5. After the fourth customer we have two $10 bill available
with us for change.

From fifth customer , we collect $20 , we cannot give the $15 change as we have two $10 bills.

Since all the customers did not receive the change correctly , the we return false. */
import java.util.*;
public class Lemonade_Change {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        System.out.print(lemonadeChange(arr));
        System.out.print(lemonadeChange2(arr));
    }
    public static boolean lemonadeChange(int[] bills) {
        //your code goes here
        HashMap<Integer, Integer> m = new HashMap<>();
        m.put(5,0); m.put(10,0); m.put(20,0);
        System.out.print(m);
        for(int x : bills){
            if(x == 5) m.put(5,m.get(5)+1);
            else if(x == 10) {
                m.put(10,m.get(10)+1);
                if(m.get(5)>=1) m.put(5,m.get(5)-1);// giving back 5 rupeees
                else return false;
            }
            else {
                 m.put(20,m.get(20)+1);
                if(m.get(5)>=1 && m.get(10)>=1) {
                    m.put(5,m.get(5)-1); m.put(10,m.get(10)-1); // this should be first
                }
                else if(m.get(5)>=3) m.put(5,m.get(5)-3); // 3 5 rupees coins
                else return false;
            }
             System.out.println(m + " for" + x);
        }
        return true;
    }
```

```
    public static boolean lemonadeChange2(int[] bills) {
    int five = 0, ten = 0;
    for (int b : bills) {
        if (b == 5) five++;
        else if (b == 10) {
            if (five == 0) return false;
            five--; ten++;
        } else {
            if (five > 0 && ten > 0) { five--; ten--; }
            else if (five >= 3) five -= 3;
            else return false;
        }
    }
    return true;
}
}
```

# striver / greedy / secduling and intervaling

## Candy.java

```
import java.util.*;

public class Candy {

    static int r[], c[], n;

    public static void main(String[] args) {

        Scanner s = new Scanner(System.in);

        int n = s.nextInt();

        int ratings[] = new int[n];

        for(int i = 0; i < n; i++) {
            ratings[i] = s.nextInt();
        }

        System.out.println("Approach 1: " + app1(ratings, n));
        System.out.println("Approach 2: " + app2(ratings));
        System.out.println("Approach 3: " + app3(ratings));
    }

    // Approach 1: Repeated checking
    // Time: O(n²) or more in worst-case analysis
    // Space: O(n)

    public static int app1(int ratings[], int n) {

        int candy[] = new int[n];

        Arrays.fill(candy, 1);

        boolean changed = true;

        while(changed) {

            changed = false;

            for(int i = 0; i < n; i++) {

                if(i > 0 && ratings[i] > ratings[i - 1]
                        && candy[i] <= candy[i - 1]) {

                    candy[i] = candy[i - 1] + 1;
                    changed = true;
                }

                if(i < n - 1 && ratings[i] > ratings[i + 1]
                        && candy[i] <= candy[i + 1]) {

                    candy[i] = candy[i + 1] + 1;
                    changed = true;
                }
            }
        }

        int sum = 0;

        for(int i = 0; i < n; i++) {
            sum += candy[i];
        }

        return sum;
    }

    // Approach 2: Recursion

    public static int app2(int ratings[]) {

        r = ratings;
        n = ratings.length;

        c = new int[n];

        Arrays.fill(c, 1);

        for(int i = 0; i < n; i++) {
```

```
            checkLeft(i);
            checkRight(i);
        }

        int sum = 0;

        for(int x : c) {
            sum += x;
        }

        return sum;
    }

    static void checkLeft(int i) {

        if(i > 0 && i < n && r[i] > r[i - 1]
                && c[i] <= c[i - 1]) {

            c[i] = c[i - 1] + 1;

            checkLeft(i + 1);
        }
    }

    static void checkRight(int i) {

        if(i >= 0 && i < n - 1 && r[i] > r[i + 1]
                && c[i] <= c[i + 1]) {

            c[i] = c[i + 1] + 1;

            checkRight(i - 1);
        }
    }

    // Approach 3: Two Arrays (Left + Right)
    // Time: O(n)
    // Space: O(n)

    public static int app3(int ratings[]) {

        int n = ratings.length;

        int left[] = new int[n];
        int right[] = new int[n];

        left[0] = 1;

        // Left to Right
        for(int i = 1; i < n; i++) {

            if(ratings[i] > ratings[i - 1]) {
                left[i] = left[i - 1] + 1;
            }
            else {
                left[i] = 1;
            }
        }

        right[n - 1] = 1;

        // Right to Left
        for(int i = n - 2; i >= 0; i--) {

            if(ratings[i] > ratings[i + 1]) {
                right[i] = right[i + 1] + 1;
            }
            else {
                right[i] = 1;
            }
        }

        int sum = 0;

        for(int i = 0; i < n; i++) {
            sum += Math.max(left[i], right[i]);
        }

        return sum;
    }
}
```

### Job_Sequencing_Problem.java

```
/*Given an 2D array Jobs of size Nx3, where Jobs[i][0] represents JobID , Jobs[i][1] represents Deadline ,
Jobs[i][2] represents Profit associated with that job. Each Job takes 1 unit of time to complete and only one job
can be scheduled at a time.

The profit associated with a job is earned only if it is completed by its deadline. Find the number of jobs and
maximum profit.

Example 1:
Input : Jobs = [ [1, 4, 20] , [2, 1, 10] , [3, 1, 40] , [4, 1, 30] ]

Output : 2 60

Explanation : Job with JobID 3 can be performed at time t=1 giving a profit of 40.

Job with JobID 1 can be performed at time t=2 giving a profit of 20.

No more jobs can be scheduled, So total Profit = 40 + 20 => 60.

Total number of jobs completed are two, JobID 1, JobID 3.

So answer is 2 60.

Example 2:
Input : Jobs = [ [1, 2, 100] , [2, 1, 19] , [3, 2, 27] , [4, 1, 25] , [5, 1, 15] ]

Output : 2 127

Explanation : Job with JobID 1 can be performed at time time t=1 giving a profit of 100.

Job with JobID 3 can be performed at time t=2 giving a profit of 27.

No more jobs can be scheduled, So total Profit = 100 + 27 => 127.

Total number of jobs completed are two, JobID 1, JobID 3.

So answer is 2 127. */
import java.util.*;
public class Job_Sequencing_Problem{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [][] = new int [n][3];
        for(int i=0;i<n;i++){
            for(int j=0;j<3;j++) arr[i][j] = s.nextInt();
        }
        System.out.print(app1(arr));
       // System.out.print(app2(arr));
    }
   /* my attempts :
    public static int app1(int arr[][]){
        int time = 1, val = 0;
        boolean not_found_job = false;
        while(!not_found_job){
            int val_max = 0;
            not_found_job = true; // defalt
        for (int i = 0;i<arr.length;i++){
            if(arr[i][1]>= time){
                not_found_job = false;
                val_max= Math.max(arr[i][2],val_max);
                //System.out.println(val_max);
            }
        }
        if(!not_found_job) {
            val+= val_max;
            System.out.println(val_max + " " + val);
            time+= 1;
        }
        else{
            break;
        }
    }

    return val;
}
public static int app2(int arr[][]){
    for (int i=0;i<arr.length;i++){
        for(int j=0;j<arr.length-1;j++){
            if(arr[j][1]>arr[j+1][1]){ // greater deadline
                int temp = arr[j][0];
                arr[j][0] = arr[j+1][0];
                arr[j+1][0] = temp;
```

```
                int temp2 = arr[j][1];
                arr[j][1] = arr[j+1][1];
                arr[j+1][1] = temp2;
                int temp3 = arr[j][2];
                arr[j][2] = arr[j+1][2];
                arr[j+1][2] = temp3;
            }
            else if(arr[j][1] == arr[j+1][1] && arr[j][2] < arr[j+1][2]){
                 // same deadline then check for value
                int temp = arr[j][0];
                arr[j][0] = arr[j+1][0];
                arr[j+1][0] = temp;
                int temp2 = arr[j][1];
                arr[j][1] = arr[j+1][1];
                arr[j+1][1] = temp2;
                int temp3 = arr[j][2];
                arr[j][2] = arr[j+1][2];
                arr[j+1][2] = temp3;
        }
    }
}
// sort ayindi
int time = 1, val = 0;
for (int x []:arr){
    System.out.print(Arrays.toString(x));
    if(x[1]>= time) {
        time += 1;
        val += x[2];
    }
}
return val;
}
}*/
// striver solution :
    // Approach 1: Greedy + Sorting
    // Time: O(n log n + n * maxDeadline)
    // Space: O(maxDeadline)
    public static int[] app1(int arr[][]) {

        int n = arr.length;

        // Sort jobs according to profit (descending)
        Arrays.sort(arr, (a, b) -> b[2] - a[2]); // idhi alavatu chesukovali

        int totProfit = 0;
        int cnt = 0;
        int maxDeadline = -1;

        // Find maximum deadline
        for(int i = 0; i < n; i++) {
            maxDeadline = Math.max(maxDeadline, arr[i][1]); // max no of days kosam kavali
        }

        // -1 means slot is free
        int hash[] = new int[maxDeadline + 1]; // plus one enduku anntey to manage zero based index , array 0 nunchi
start ayina , we conisider days form 1 enduku anntey day 0 unndadu kabatti

        Arrays.fill(hash, -1); // kali slots

        // Schedule jobs
        for(int i = 0; i < n; i++) {
            // Find latest available slot
            for(int j = arr[i][1]; j >= 1; j--) { //start ee deadline unna riju nunchi chesaru
                if(hash[j] == -1) {
                    cnt++;
                    hash[j] = arr[i][0];
                    totProfit += arr[i][2];
                    break;
                }
            }
        }
        return new int[]{cnt, totProfit};
    }
}
//https://claude.ai/share/e78d01ea-2de9-41b8-8472-432bb4337cd9
```

### N_Meetings.java

```
/*Given one meeting room and N meetings represented by two arrays, start and end, where start[i] represents the
start time of the ith meeting and end[i] represents the end time of the ith meeting, determine the maximum number of
meetings that can be accommodated in the meeting room if only one meeting can be held at a time. A meeting starting
at the same time another meeting ends is considered overlapping.

Example 1:
Input : Start = [1, 3, 0, 5, 8, 5] , End = [2, 4, 6, 7, 9, 9]

Output : 4

Explanation : The meetings that can be accommodated in meeting room are (1,2) , (3,4) , (5,7) , (8,9).

Example 2:
Input : Start = [10, 12, 20] , End = [20, 25, 30]

Output : 1

Explanation : Given the start and end time, only one meeting can be held in meeting room.

Now Your Turn!
Pick the correct output for the given input
Input : Start = [1, 4, 6, 9] , End = [2, 5, 7, 12]


2

4

1

3
Still unsure what the problem is asking ?

Let’s go through a few more examples, step by step, to make it clearer.

Constraints:
1 <= N <= 105
0 <= start[i] < end[i] <= 105*/
import java.util.*;

public class N_Meetings {
    public static void main(String[] args) {

        Scanner s = new Scanner(System.in);

        int n = s.nextInt();

        int start[] = new int[n];
        int end[] = new int[n];

        for(int i = 0; i < n; i++) {
            start[i] = s.nextInt();
        }

        for(int i = 0; i < n; i++) {
            end[i] = s.nextInt();
        }

        System.out.println(app1(start, end, n));
    }

    // Approach 1: Bubble Sort + Greedy
    // Time: O(n²), Space: O(1)
    public static int app1(int start[], int end[], int n) {

        // Sort according to ending time
        for(int i = 0; i < n - 1; i++) {

            boolean swapped = false;

            for(int j = 0; j < n - 1 - i; j++) {

                if(end[j] > end[j + 1]) {

                    int temp = end[j];
                    end[j] = end[j + 1];
                    end[j + 1] = temp;

                    temp = start[j];
                    start[j] = start[j + 1];
                    start[j + 1] = temp;
```

```
                    swapped = true;
                }
            }

            if(!swapped) break;
        }

        int count = 0;
        int lastEnd = -1;

        // Select maximum non-overlapping meetings
        for(int i = 0; i < n; i++) {

            if(start[i] > lastEnd) {

                count++;
                lastEnd = end[i];
            }
        }

        return count;
    }
}
```

### Non_Overlapping_Intervals.java

```
//package greedy.secduling and intervaling;

public class Non_Overlapping_Intervals {

}
```

### Shortest_Job_First.java

```
//package greedy.secduling and intervaling;
/*A software engineer is tasked with using the shortest job first (SJF) policy to calculate the average waiting time
for each process. The shortest job first also known as shortest job next (SJN) scheduling policy selects the waiting
process with the least execution time to run next.

You are given an array of integers bt of size n representing the burst times (execution times) of n processes.

Your task is to calculate the average waiting time for all processes when scheduled using the SJF policy. The
waiting time of a process is the total time a process has to wait before its execution starts, which is the sum of
burst times of all previously executed processes.

Return the floor of the average waiting time, i.e., the largest whole number less than or equal to the actual
average.

Example 1:
Input : bt = [4, 1, 3, 7, 2]

Output : 4

Explanation : The total waiting time is 20.

So the average waiting time will be 20/5 => 4.

Example 2:
Input : bt = [1, 2, 3, 4]

Output : 2

Explanation : The total waiting time is 10.

So the average waiting time will be 10/4 => 2. */
import java.util.*;
public class Shortest_Job_First {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);

        int n = s.nextInt();
        int arr [] = new int[n];
        for (int i=0;i<n;i++) arr[i] = s.nextInt();
        System.out.print(app1(arr));
    }
    public static long app1(int[] bt) {
        //your code goes here
        Arrays.sort(bt);
        double sum = 0;
```

```
        double wt = 0;
        for(int x : bt){
            wt += sum;
            sum += x;
            System.out.print(x + " "+ sum+ " ");
            System.out.println(x + " "+ wt);
        }
        return (long) Math.floor(wt/bt.length);
    }

}
//https://claude.ai/share/9ae557f1-65dd-4183-9324-75573cd706aa
```

# striver / patterns

### p6.java

```
/* */
```

### pattern1.java

```
/*Given an integer n. You need to recreate the pattern given below for any value of N. Let's say for N = 5, the
pattern should look like as below:
*****
*****
*****
*****
*****
*/
public class pattern1{
    public static void main(String args[]){
        int n=5;
        app_2(n);
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                System.out.print("*");
            }
            System.out.println();
        }
    }
    public static app_2(int n){
        int i = 0;
        while(i<=n){
            if(i!= 0 && i!=1 &&  n%i== 0){
                System.out.println();
            }
            System.out.print("*");

        }
    }
}
```

### pattern2.java

```
/*Given an integer n. You need to recreate the pattern given below for any value of N. Let's say for N = 5, the
pattern should look like as below:



*

**

***

****

*****



Print the pattern in the function given to you. */
public class pattern2{
    public static void main(String args[]){
        int n = 5;
        for(int i=0;i<n;i++){
            for(int j=0;j<=i;j++){
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

### pattern3.java

```
/*Given an integer n. You need to recreate the pattern given below for any value of N. Let's say for N = 5, the
pattern should look like as below:



1

12

123

1234

12345



Print the pattern in the function given to you. */
public class pattern3{
    public static void main(String args[]){
        int n = 5;
        for(int i=1;i<=n;i++){ // zero pedthe oka extra blank vastadi
            for(int j=0;j<i;j++){
                System.out.print(j+1);
            }
            System.out.println();
        }
    }
}
```

### pattern4.java

```
//13-08-2026
/*Given an integer n. You need to recreate the pattern given below for any value of N. Let's say for N = 5, the
pattern should look like as below:



1

22

333

4444

55555 */
import java.util.*;
public class pattern4{
    public static void main(String ars[]){
        int n =5;
        app_2(n);
        for(int i=1;i<=n;i++){
            for(int j =0;j<i;j++){
                System.out.print(i);
            }
            System.out.println();
        }

    }
    //deni time :  O(n^2) and space O(1)
    public static void app_2(int n){
        for(int i=0;i<n;i++){
            StringBuilder sb = new StringBuilder();
            for(int j=0;j<=i;j++){
                sb.append(i+1);
            }
            System.out.println(sb);
        }
    }
    //deni time kuda O(n^2) ee kani only one print statement tho ney aypiothadi
}
//other best approches ai lo unnai
```

### pattern5.java

```
/*Given an integer n. You need to recreate the pattern given below for any value of N. Let's say for N = 5, the
pattern should look like as below:



*****

****

***

**

*



Print the pattern in the function given to you. */
public class pattern5{
    public static void main(String args[]){
        int n = 5;
        for(int i=n;i>0;i--){
            for(int j=i;j>0;j--){
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

# striver / rec

### factorial.java

```
/*You are given an integer n. Return the value of n! or n factorial.



Factorial of a number is the product of all positive integers less than or equal to that number.


Example 1

Input: n = 2

Output: 2

Explanation: 2! = 1 * 2 = 2.

Example 2

Input: n = 0

Output: 1

Explanation: 0! is defined as 1.*/
import java.util.*;
public class factorial {
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        if(n<0){
            return;
        }
        System.out.println(fact(n));
    }
    public static long fact(int n ){
        if(n == 0 || n == 1){
            return 1;

        }
        return  n *fact(n-1);
    }
}
// 16- 9- 26 : https://claude.ai/share/cf1ccf75-ebd1-47b4-8842-a43c3d9ea05d
```

### printlinearly.java

```
public class printlinearly{
    public static void main(String args[]){
        do_linear(0);
    }
    public static void do_linear(int i){
        if(i == 4){
            return ;
        }
        System.out.println(i);
        do_linear(i+1);
    }
}
```

### printlinearlybacktracking.java

```
public class printlinearlybacktracking{
    public static void main(String args[]){
        int i=3,n = 3;
        do_print(i,n);
    }
    public static void do_print(int i,int n){
        if(i<=0){
            return;
        }
        do_print(i-1,n); // 1st recurrsive call and then the statement
        System.out.println(i);
    }
}
```

### printname.java

```
public class printname{
    public static void main(String args[]){
        do_print("shashi"); // static vadatam anta manchidi kadu annta
        do_print2("shashi",0,3); // idhi chala manchidi annta

    }
    static int c=0;
    public static void do_print(String name){
         if(c == 3){
            return;
        }
        c++;
        System.out.println(name);
        do_print(name);
    }
    public static void do_print2(String name,int i,int n){
        if(i==n){
            return ;
        }
        i++;
        System.out.println(name);
        do_print2(name,i,n);
    }
}
```

### printulta.java

```
/*Given an integer n, write a function to print all numbers from n to 1 (inclusive) using recursion.

You must not use any loops such as for, while, or do-while.
The function should print each number on a separate line, in decreasing order from n to 1

Example 1

Input: 5

Output:

5

4

3

2

1

Example 2

Input: 1

Output:

1

Constraints

1 <= n <= 100*/
public class printulta{
    public static void main(String args[]){
        do_print(5);
    }
    public static void do_print(int n){
        if(n == 0){
            return;

        }
        System.out.println(n);
        do_print(n-1);
        System.out.println("shashi"); // just to make u understand how recusrrion works
    }
}
```

### printulta2.java

```
public class printulta2{
    public static void main(String args[]){
        int i=0;int n = 3;
        do_print(i,3);
    }
    public static void do_print(int i,int n){
        if(i>=n){
            return;
        }
        do_print(i+1,n);
        System.out.println(i+1);
    }
}
```

### reverse_of_array.java

```
/*Given an array arr of n elements. The task is to reverse the given array. The reversal of array should be inplace.


Example 1

Input: n=5, arr = [1,2,3,4,5]



Output: [5,4,3,2,1]



Explanation: The reverse of the array [1,2,3,4,5] is [5,4,3,2,1]

Example 2

Input: n=6, arr = [1,2,1,1,5,1]



Output: [1,5,1,1,2,1]



Explanation: The reverse of the array [1,2,1,1,5,1] is [1,5,1,1,2,1]. */
import java.util.*;
public class reverse_of_array{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        if(n <= 0){
            return;
        }
        int arr[] = new int [n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        System.out.println(Arrays.toString(rev(arr,0,n-1)));
    }
    public static int [] rev(int arr [],int i,int j){
        if(i>=j){
            return arr;
        }
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        return rev(arr,i+1,j-1);
    }
}
//https://claude.ai/share/65ae5ab2-23c3-4e10-922e-4e127ea8f858
// 16-9-26
```

### sum_of_nterms.java

```
/*Given an integer N, return the sum of first N natural numbers. Try to solve this using recursion.


Example 1

Input : N = 4

Output : 10
```

```
Explanation : first four natural numbers are 1, 2, 3, 4.

Sum is 1 + 2 + 3 + 4 => 10.

Example 2

Input : N = 2

Output : 3

Explanation : first two natural numbers are 1, 2.

Sum is 1 + 2 => 3. */
public class sum_of_nterms{
    public static void main(String args[]){
        int n = 4; // if u want take input later
        int i=0,sum = 0;
        do_sum(n,i,sum);
        System.out.println(do_sum2(n));
        System.out.println(do_sum3(n));


    }
    static int sum2 = 0;
    public static void do_sum(int n,int i, int sum){
        if(i==n){
            System.out.println(sum);
            return;
        }
        i++;
        sum = sum + i;
        do_sum(n,i,sum);
    }

    public static int do_sum2(int n){
        if(n<=0){
            return sum2;
        }
        sum2 += n;
        return do_sum2(n - 1);
    }
    public static int do_sum3(int n){
        if(n==0){
            return 0;
        }
        return n + do_sum3(n-1);
    }

}
```

# striver / sortings

### bubblesort.java

```
/*Given an array of integers called nums,sort the array in non-decreasing order using the bubble sort algorithm and
return the sorted array.



A sorted array in non-decreasing order is an array where each element is greater than or equal to all preceding
elements in the array.


Example 1

Input: nums = [7, 4, 1, 5, 3]

Output: [1, 3, 4, 5, 7]

Explanation: 1 <= 3 <= 4 <= 5 <= 7.

Thus the array is sorted in non-decreasing order.

Example 2

Input: nums = [5, 4, 4, 1, 1]

Output: [1, 1, 4, 4, 5]

Explanation: 1 <= 1 <= 4 <= 4 <= 5.

Thus the array is sorted in non-decreasing order.*/
import java.util.*;
public class bubblesort{
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int arr [] = new int[n];
        for(int i=0;i<n;i++) arr[i] = s.nextInt();
        System.out.println(Arrays.toString(bubble_sort(arr,n)));
    }
    public static int [] bubble_sort(int arr [],int n){ // direct array return chestunnam kada anudkey int []

        for(int i = 0;i<n;i++){
            for(int j = 0;j<n-1;j++){
                if(arr[j+1]<arr[j]){
                    int temp = arr[j+1];
                    arr[j+1] = arr[j];
                    arr[j] = temp;
                }
            }
        }
        return arr;
    }
}
```

### selectionsort.java

```
/*Given an array of integers nums, sort the array in non-decreasing order using the selection sort algorithm and
return the sorted array.
A sorted array in non-decreasing order is an array where each element is greater than or equal to all previous
elements in the array.
Example 1
Input: nums = [7, 4, 1, 5, 3]
Output: [1, 3, 4, 5, 7]

Explanation: 1 <= 3 <= 4 <= 5 <= 7.

Thus the array is sorted in non-decreasing order.

Example 2

Input: nums = [5, 4, 4, 1, 1]

Output: [1, 1, 4, 4, 5]

Explanation: 1 <= 1 <= 4 <= 4 <= 5.

Thus the array is sorted in non-decreasing order. */
import java.util.*;
```

```
public class selectionsort{
    public static void main(String args[]){
        int min_index = 0;
        int a[] = {1,5,4,6,7};
        int n = a.length;
        for(int i=0;i<n;i++){
            min_index = i;
            for(int j=i+1;j<n;j++){
                if(a[min_index]>a[j]){
                    min_index  = j;
                }
            }
            //swap :
            int temp = a[i];
            a[i]=a[min_index];
            a[min_index]=temp;
        }
        System.out.println(Arrays.toString(a)); //util lo unntadi annta
    }
}
//https://claude.ai/share/f0c2fae6-ef09-4db6-bc80-d8eec1338e8a
```